

What are Web Services?

Introduction

A web service is a method for sending messages between systems or performing a query using the HTTP protocol (as used by your web browser to retrieve this page) as a basis. This makes it easy to design a protocol able to send a message to any connected server. The approach also makes it easy to integrate standard HTTP features (including security and authentication) into the protocol. There are many communication protocols designed around a web service architecture. In this module, we will be focusing on the two most common, REST and SOAP. In recent years, many standards have been developed that make use of various web service protocols. These include a number of recent health related standards, including [FHIR\(opens in a new tab\)](#) and [XDS\(opens in a new tab\)](#).

A summary of REST and SOAP follows. The two approaches will be discussed in more detail later in this module.

- REST: Representational State Transfer. A philosophy for designing web services that uses various features of the HTTP protocol, such as methods and headers, to describe the resource to be accessed and the action to be taken.
- SOAP: Simple Object Access Protocol. An XML protocol built on top of HTTP that uses request response semantics to make function calls between machines.

The content of a message sent via a web service is generally XML. Message formats that rely on specific character encoding and line ending characters (such as HL7) should not be transmitted raw over a web service connection. Attempting to transmit such a message raw can result in message corruption.

Further Reading

A wealth of content exists online relating to web services and their related protocols. Learners seeking additional information might find the following links useful.

- [http://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol\(opens in a new tab\)](http://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol(opens in a new tab))
- [http://en.wikipedia.org/wiki/Web_service\(opens in a new tab\)](http://en.wikipedia.org/wiki/Web_service(opens in a new tab))

SOAP vs REST

SOAP and REST are two approaches to solving the same basic problem of performing a task on a remote system, such as retrieving a patient record or ordering a radiology test. In many cases, the designer of a web service could use either REST or SOAP to create the service, with the decision made on the basis of a non-technical preference. This has resulted in numerous discussions on the relative virtues of SOAP and REST. When using

Rhapsody to integrate with a system that only supports one protocol, such a discussion will be moot.

For anyone who is interested in more background on these protocols and the differences between them, the following links provide a useful start point.

- <http://blog.smartbear.com/apis/understanding-soap-and-rest-basics/>(opens in a new tab)
- <https://msdn.microsoft.com/en-us/magazine/dd942839.aspx>(opens in a new tab)
- <http://nordicapis.com/rest-vs-soap-nordic-apis-infographic-comparison/>(opens in a new tab)

Web Services in Rhapsody

Rhapsody provides support for SOAP web services as a client and as a host via the **Web Service Client** and **Web Service Hosting** communication points. Support for REST web services is provided by two different groups of communication points (based on the style of REST web service). These groups are the **HTTP Client** and **HTTP Server** communication points and the **JavaScript REST Client** communication point. This module covers all of these communication points.

The following table summarizes the different types of web service communication points supported by Rhapsody.

	Client	Server
HTTP		
JavaScript REST		
SOAP Web Services		

Rhapsody also provides a number of REST interfaces that allow external applications to query and interact with the engine. These are not covered as part of Rhapsody Professional. They are documented in the Rhapsody Documentation here: [REST API](#)(opens in a new tab) and covered in Rhapsody Expert.

Understanding REST

Introduction

A REST web service is a philosophy rather than a fixed protocol for designing web services. REST web services use the various attributes of the HTTP protocol to define records, formats, and actions performed by the web service. A REST web service will generally use the following design guidelines:

- The record to be accessed (for read, update or delete operations) is contained in the URL.
- The HTTP method is used to define the action to be performed. Many systems will use CRUD semantics to define the appropriate HTTP method to use for each action. CRUD stands for create, read, update and delete.
- The HTTP accept header is used to provide the server with details of the message format desired by the client system.

The emerging HL7 FHIR standard makes use of REST style communications for transmission of messages.

Examples

Following are examples of some existing REST APIs

- HL7 FHIR: <https://www.hl7.org/fhir/http.html>(opens in a new tab)
- Amazon S3: <https://docs.aws.amazon.com/AmazonS3/latest/API/Welcome.html>(opens in a new tab)
- Atlassian Confluence: <https://docs.atlassian.com/atlassian-confluence/REST/latest/>(opens in a new tab)
- Tesla Model S: <https://tesla-api.timdorr.com/>(opens in a new tab)

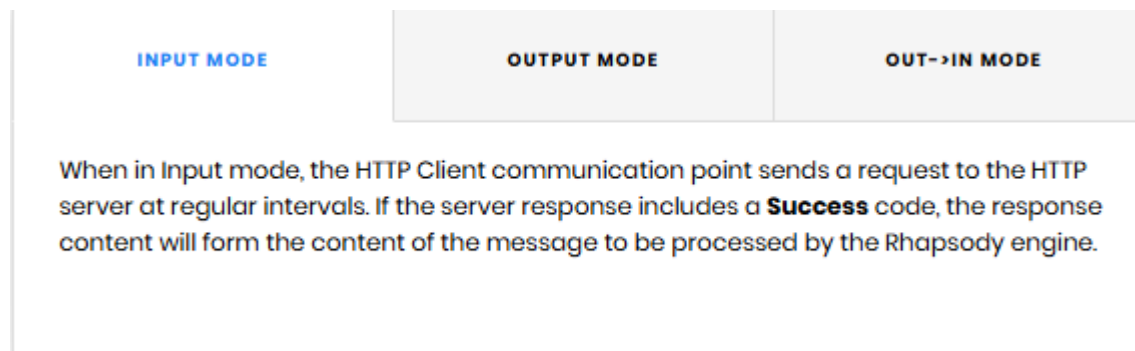
Further Reading

- http://en.wikipedia.org/wiki/Representational_state_transfer(opens in a new tab)
- <https://code.tutsplus.com/a-beginners-guide-to-http-and-rest--net-16340t>(opens in a new tab)
- <https://github.com/Microsoft/api-guidelines/blob/vNext/Guidelines.md>(opens in a new tab)
- <http://fhirblog.com/2013/10/15/a-rest-primer-fhir-style-part-i/>(opens in a new tab)
- https://en.wikipedia.org/wiki/Create,_read,_update_and_delete

HTTP Client and Server

HTTP Client

The **HTTP Client** communication point connects to an HTTP resource from Rhapsody. While the URL of the resource may be specified via a message property or Rhapsody Environment Variable, only a single HTTP connection method can be used per communication point. This, when combined with the supported operation modes, makes this communication point very good when performing ad-hoc HTTP requests, but less suited when communicating with a REST service.



Communication Point Properties

Refer to the Documentation for a detailed description of the configuration options for this communication point.

Connecting Through a Proxy Server

Where the connection to the server is being made through a proxy server, the **Proxy Type**, **Proxy Hostname** and **Proxy Port** settings can be used to define the proxy server. Rhapsody supports basic and digest HTTP authentication, which can be specified via the **Proxy HTTP Authentication Username** and **Proxy HTTP Authentication Password** properties.

Published Properties

The HTTP Client communication point in Input mode includes the following published properties:

- Response Code - the status code from the HTTP response status line of the HTTP response which returned this message.
- Response Message - the reason phrase from the HTTP response status line of the HTTP response which returned this message.
- Content Type - the MIME type of the HTTP response body, which is contained in the body of the returned message.

- **Content Encoding** - the MIME encoding (character set) of the HTTP response body, which is contained in the body of the returned message.

HTTP response headers are added to the message as message properties with a http: prefix. For example, the Server response header would be added as the http:Server message property, while a X-Powered-By header would be added as the http:X-Powered-By message property.

HTTP Server

The HTTP Server communication point enables Rhapsody to act as a web server so HTTP clients such as a REST Client or an application using CURL can directly exchange messages with Rhapsody. This communication point is normally configured to listen for HTTP requests to a specific URL on a specific port and to perform a specific action when the request arrives.

The communication point can be configured to perform different actions depending on the request method. This makes it straightforward to implement a REST service in Rhapsody.

Communication Point Properties

Supported HTTP Actions

- **Receive Message** - (Input or In->Out mode only) Receives a message from the request by reading its associated HTTP content.
- **Receive Message in Query Parameters** - (Input or In->Out mode only) Receives a message from the **Request Parameters** and **Message Body Parameter Name** configuration parameters.
- **Return Message** - (Output mode only) All request parameters and content will be ignored. The response includes the next queued message along with a 200 response code. If there is no queued message and none arrives within a specified period, a Gateway Timeout (504) will be returned.
- **Return Static Content** - All request parameters and content will be ignored. The static content specified by the **Static Content** parameter will be returned along with a 200 response code.
- **Redirect to RedirectURL** - Redirects the request to the URL specified by the **Redirect URL** configuration property.

Action Not Supported is returned when the communication point does not support a request method. When this occurs, a 405 (Method Not Supported) response will be returned.

Published Properties

The HTTP Server communication point in Input mode includes the following published properties:

- **ContentType** - the MIME content type of the Rhapsody message received in the content of the HTTP request. Usually will be text/plain or text/xml.
- **RequestMethod** - the HTTP request method that was used to make the request to the HTTP Server communication point. Typically POST , GET , PUT , or DELETE.
- **ServerURL** - the URL of this HTTP Server communication point that messages were received from.
- **http:request-line** - the request line that was used to make the request to the HTTP Server communication point.
- **http:request-url** - the URL that was requested in the request to the HTTP Server communication point.

HTTP request headers are added to the message as message properties with a http: prefix. For example, the user-agent request header would be added as the http:user-agent message property.

Exercise - HTTP Client

Introduction

In this exercise, we will connect to a REST style HTTP service that implements a simple patient management system. The service is self-documenting. Browsing to the REST service using a web browser will by default display details of the various methods that can be used. Start by browsing to <http://resources.edu.rhapsody.global/professional/services/REST/patient/> (opens in a new tab) to observe how the REST API accepts requests.

Next, we will use Rhapsody to get a patient record from the REST web service.

Create a HTTP Client Communication Point

Create a new HTTP Client communication point. The communication point must be configured in Out->In mode. From the Configuration tab, make the following configuration changes:

- **URL** - \$REST-URL
- **HTTP Method** - GET
- **Message Content** - None
- **On 4xx Error Response** - Return error content as input

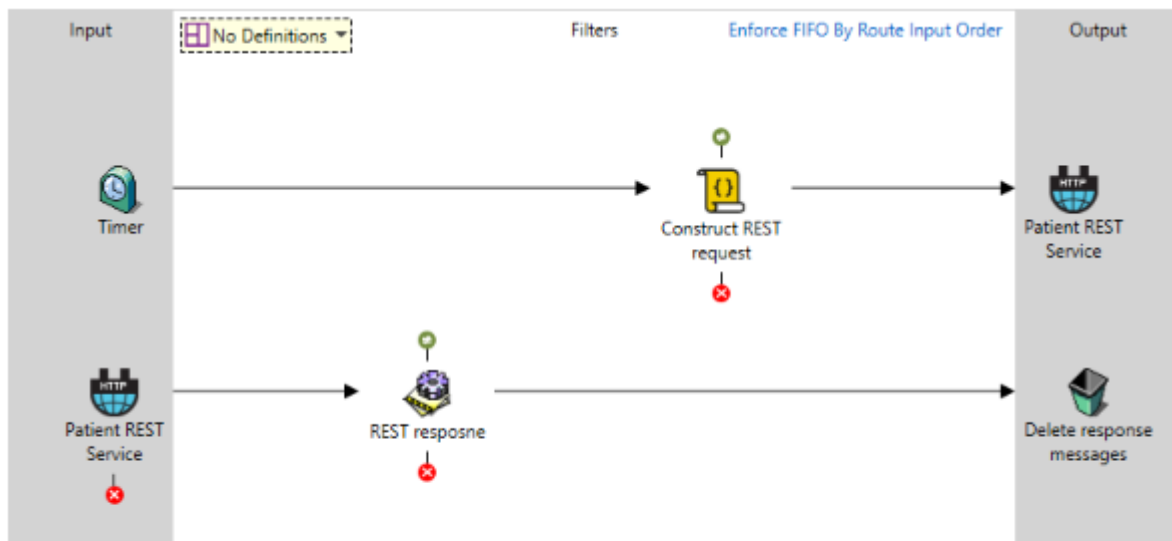
Property	Value
URL *	\$REST-URL
HTTP Method *	GET
Request Headers	
Follow Redirects *	☒
Use HTTPS *	☐
SSL Protocol Mode *	System
Specify Exact TLS Version *	
SSL Cipher Suites *	
Trusted Certificates	
Secure Keys	
Hostname Verification	☐
HTTP Authentication Username	
HTTP Authentication Password	
Protocol Support *	Standard HTTP only
Read Timeout (ms) *	10000
Connect Timeout (ms) *	60000
Proxy Type	System
Proxy Hostname	
Proxy Port	
Proxy HTTP Authentication Username	
Proxy HTTP Authentication Password	
Refresh Rate *	60000
Set If-Modified-Since Header	☐
Response Headers	
Message Content *	None
Content Type	
Content Encoding	
WWW Form Encoding Scheme	
Definition File	
Additional Request Headers	
Request Parameters	
Form Names and Values	
On 4xx Error Response *	Return error content as input (for Out->In mode only)
On 5xx Error Response *	Mark as connection failed
Dynamic Connection Failed Action *	Treat as message error
Copy headers	☐
Regular expressions	

The **URL** parameter will be set to the correct value in a JavaScript filter we will create below. The **Message Content** option needs to be set to None as a GET request is not able to send the message body as part of the HTTP request. The **On 4xx Error Response** option is set to return the error message to the route so that we can inspect the error messages generated by the REST server.

Create a Route

Create a new Route. Create and add a Timer communication point to the input of the route. This will be used to seed messages into the configuration. The HTTP Client communication point should be added to both the input and output of the route. Finally, create a Sink communication point and add it to the output of the route. The Timer communication point should be linked to the HTTP Client communication point via a JavaScript (v.2) filter. Responses from the HTTP Client communication point should be

sent to a No-operation filter that then sends messages to the Sink communication point.



The JavaScript filter should be configured as follows:

```
// Create the output message
```

```
const next = output.append(input[0]);
```

```
// Set of patient identifiers to test against the web service
```

```
const patientIdList = ['84568-4564', '84502-7664', '87619-9902', '84568-4341', '84568-3168', '91037-0197'];
```

```
// Hostname where the patient REST web service is hosted. Normally this would
```

```
// be defined as an environment variable and loaded into JavaScript via the
```

```
// getVariable() function.
```

```
const hostname =
```

```
'http://resources.edu.rhapsody.global/professional/services/REST/patient/';
```

```
// Patient id to search for. The incCounter() function is used to iterate over
```

```
// the set of patient identifiers.
```

```
const patientId = patientIdList[incCounter('Web Service Testing') % 6];
```

```
// Define message property to hold the REST URL
```

```
next.setProperty('REST-URL', hostname + patientId);
```

Testing the Web Service

Check in, then start the configuration you have just created. Open the Management Console and inspect a range of messages output from the HTTP Client communication point you created. Select the **Response received by message tracking** option to view the response associated with each message sent. One of the patient identifiers in the patientIdList in the JavaScript does not match a patient in the web service. Can you work out which identifier it is?

Message Properties

Output Message

Property Name	Property Value
InputTime	1563362320198
REST-URL	http://resources.edu.rhapsody....
rhapsody:InputCommunicationPoint	Timer
rhapsody:InputCommunicationPoi...	183

Showing 4 of 4

Response Message

[View](#)

Property Name	Property Value
ContentEncoding	NULL
ContentType	text/xml; charset=utf-8
http:Cache-Control	no-store, no-cache, must-revali...
http:Connection	keep-alive
http:Content-Length	482
http:Content-Security-Policy	default-src 'self' style-src 'self' '...
http:Content-Type	text/xml; charset=utf-8
http:Date	Wed, 17 Jul 2019 11:18:42 GMT
http:Pragma	no-cache
http:Referrer-Policy	same-origin

Showing 10 of 23

Message Body

Tree View

Flat Wire

Output Message

Show Whitespace Encoding: windows-1252

Size: 0 bytes



Download

Response Message

[View](#)

XML Encoding: windows-1252

Size: 482 bytes

```
<patient>
  <identifier type="MRN">
    84568-4341
  </identifier>
  <name>
    <givenName>
      Max
    </givenName>
  </name>
</patient>
```

Download

Extension - Delete

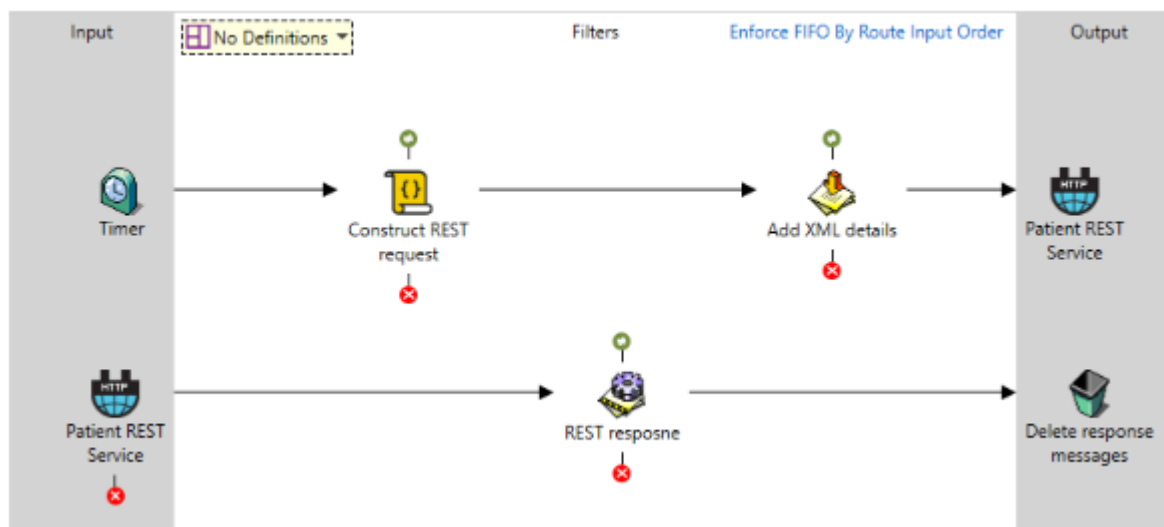
The test REST web service also allows us to add, update and delete records. Update the HTTP Client communication point as follows to test the delete function of the web service:

- **HTTP Method - DELETE**

Send some test messages to the server. Use the Management Console to inspect responses from the web service to determine how the various requests were processed.

Extension - Add and Update

Sending a PUT request to the test REST service containing an XML message performs an either an update operation (if a patient id is included in the URL) or add operation (if a patient id is not included). The [information\(opens in a new tab\)](#) page for the service tells us that the XML in the message is similar to the XML messages generated by a GET request, with name field required and id field ignored.



Configure the filter by setting the Body Template File property with an XML file that will be understood by the web service, for example:

Add XML Details Properties

General Configuration **Message Collector** Notes Support Notes Auxiliary Files

Properties with an asterisk (*) are required. [Test Filter Configuration](#)

Property	Value
Maximum Concurrency	0
Definition	
Body Template File	add.xml (New)
Field Mappings *	

Help OK Cancel

Update the HTTP Client communication point as follows:

- **HTTP Method** - PUT
- **Message Content** - Message Body

Changing the **Message Content** property will place the content of the Rhapsody message into the request sent to the web service.

Send some test messages to the server. Use the Management Console to inspect responses from the web service to determine how the various requests were processed.

Extension - Process Errors

We can detect errors returned from the web service by looking at the status code of the response. The documentation page for the test web service tells us that any response with a code in the 400 range is an error. Add a conditional connector to the response processing logic to filter out all error messages and send them to a Directory communication point. In a production environment, these error messages would normally be sent to a sending system or an administrator.

Hint: you want to look at the ResponseCode message property that is populated by the HTTP Client communication point with the response code from the web service.

Exercise - HTTP Server

Introduction

In this exercise, we will create a REST style HTTP server that when queried returns information associated with a result in a lookup table. The SexCode lookup table created for the lookup table exercise in The *Rhapsody Services* Module will be reused here. Start by creating a new folder in the Rhapsody Workspace called **Sex Code Lookups**, then create an HTTP Server communication point and a JavaScript (v.2) filter in the folder using the instructions below.

Create a HTTP Server Communication Point

Create an HTTP Server communication point running in In->Out mode. The communication point should be configured as follows:

- **Local Port** - 8110
- **Context Path** - /SexCodes
- **Get Action** - Receive message in query parameters
- **Content Type** - text/xml

Sex code REST Lookup Properties

General Connection Management Configuration **Scheduling** Message Tracking Notes Support Notes Auxiliary Files

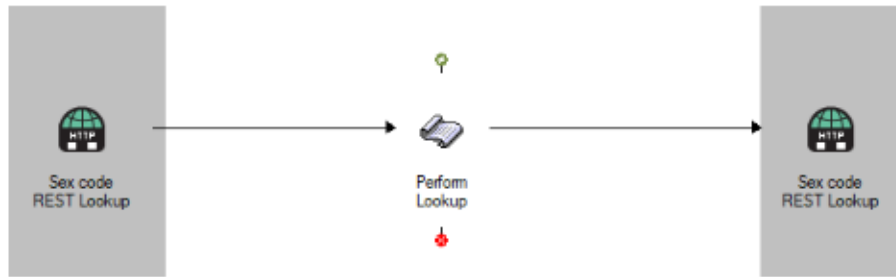
Connection Mode:

Properties with an asterisk (*) are required.

Property	Value
Local Address	
Local Port *	8110
Context Path *	/SexCodes
Timeout *	0
Use HTTPS *	☐
SSL Protocol Mode *	
SSL Cipher Suites *	
Require Client Authentication *	☐
Secure Keys *	
Trusted Certificates *	
Get Action *	Receive message in query parameters
Put Action *	Action not supported
Post Action *	Action not supported
Head Action *	Action not supported
Trace Action *	Action not supported
Delete Action *	Action not supported
Options Action *	Action not supported
Redirect URL	
Static Content	
Request Parameters	
Message Body Parameter Name	
Allow Null Message Body	☐
Protocol Support	Standard HTTP only
Request Headers	
Content Type	text/xml
Content Encoding	
Response Headers	
Copy headers	☐
Regular expressions	

Create a Route

Create a new Route. Add the HTTP Server communication point created above to both the input and output of the route, and link together with a JavaScript filter.



The JavaScript filter should be configured as follows:

```
// Create the output message
const next = output.append(input[0]);

// Get request URL from message
const request = next.getProperty('http:request-url');

// Split URL on forward slashes to find IEC code using JavaScript split function
const queryCode = request.split('/').at(-1);

// Perform code lookup
const sexLookup = lookup('SexCodes', 'IEC5218', queryCode);

// Build output message
if (sexLookup != null) {
    // Lookup succeeded, populate XML message with code and description from lookup
    table
    next.xml = '<sex><code/><description/></sex>';
}
```

```
next.setField('/sex/code', sexLookup.Code);
next.setField('/sex/description', sexLookup.Description);

// Set a HTTP status of 200 (record found)
next.setProperty('ResponseCode', '200');
} else {
    // Lookup failed, return an error message
    next.xml = '<sex><error>Unknown code</error></sex>';

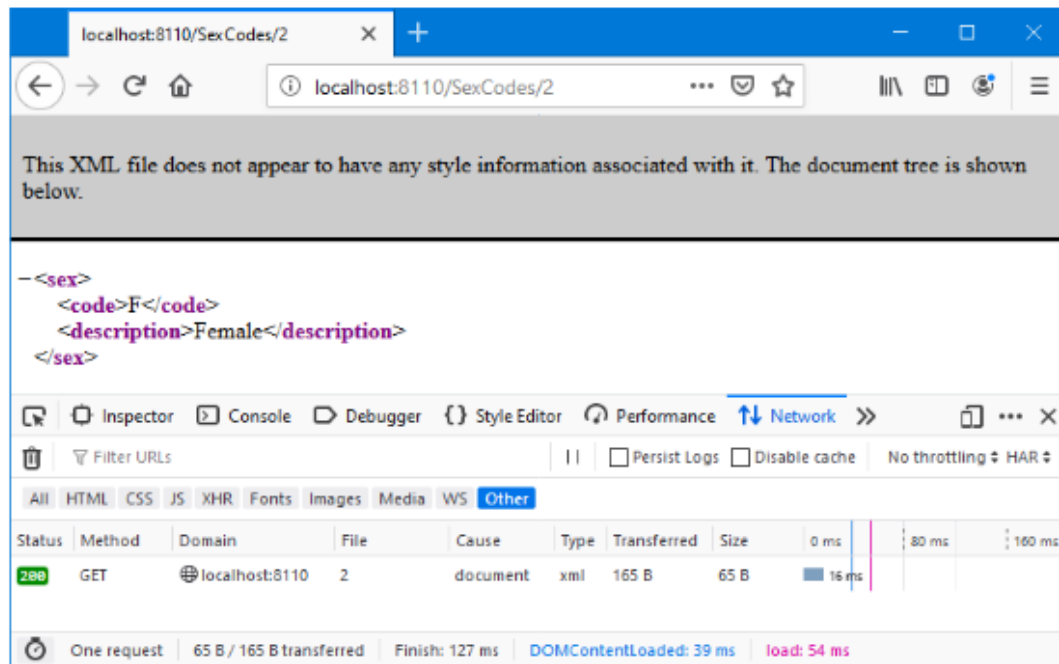
    // Set a HTTP status of 404 (record not found)
    next.setProperty('ResponseCode', '404');
}
```

Testing the Web Service

Check in, then start the configuration you have just created.

Open a web browser and point it to the following

URL: <http://localhost:8110/SexCodes/2>(opens in a new tab). If Rhapsody is running on a different machine than your web browser, the hostname will need to be changed appropriately. In a web browser such as Firefox, something similar to the following will be returned. If you view the network monitor built into the browser, you will see that the response code for the message is 200, which indicates that the code has been returned.



For reference, the contents of the SexCodes lookup table is presented below.

IEC5218	Code	Description
0	U	Unknown
1	M	Male
2	F	Female
9	N	Not applicable

Inspect the Management Console to see what information was sent to Rhapsody and how the response was constructed. Notice all the properties that start with http:, these contain the various HTTP headers sent with the request. In a more complex solution, you might need to inspect these so that the request can be correctly processed.

Property Name	Property Value
ContentType	NULL
http:accept	text/html,application/xhtml+xml...
http:accept-encoding	gzip, deflate
http:accept-language	en-US,en;q=0.5
http:cache-control	max-age=0
http:connection	keep-alive
http:host	localhost:8110
http:request-line	GET /SexCodes/2 HTTP/1.1
http:request-url	http://localhost:8110/SexCod...
http:upgrade-insecure-requests	1
Show More	

Showing 10 of 16

Message Body ▾

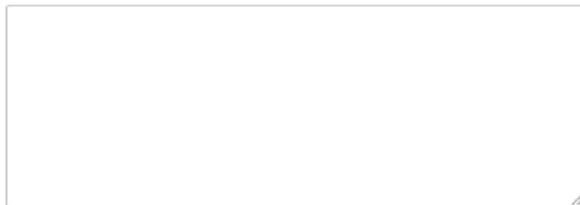
Tree View ▸

Flat Wire ▾

In

Show Whitespace ▾ Encoding: windows-1252

Size: 0 bytes



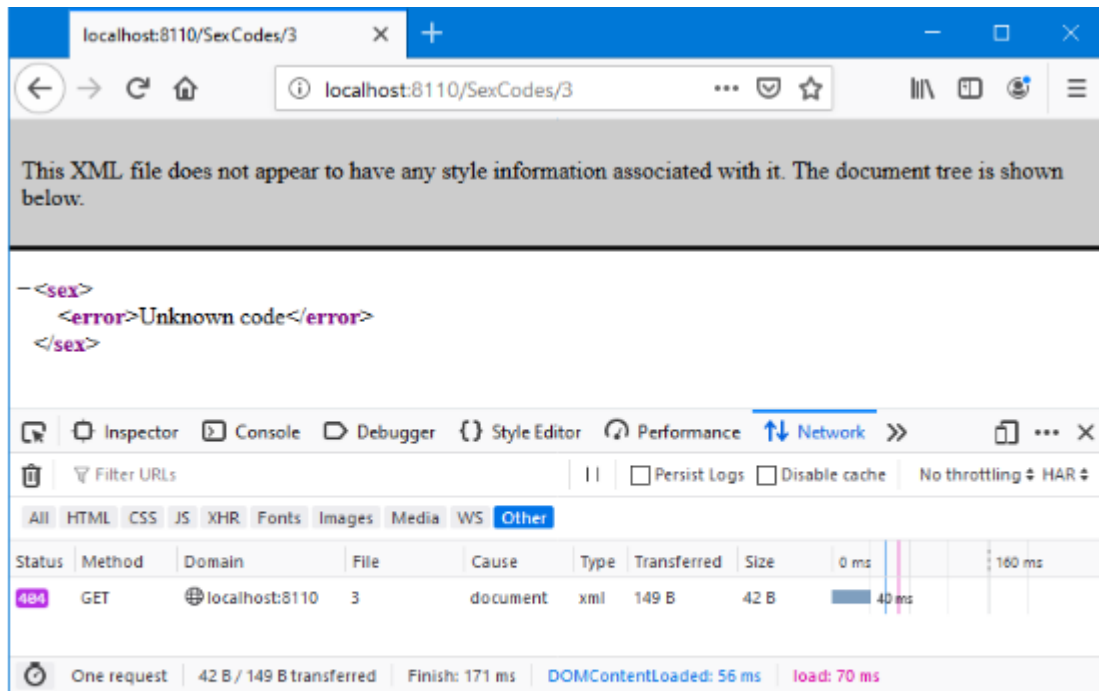
Out

XML ▾ Encoding: UTF-8

Size: 65 bytes

```
<sex>
  <code>
  F
  </code>
  <description>
  Female
  </description>
</sex>
```

Now modify the URL in your browser to <http://localhost:8110/SexCodes/3> (opens in a new tab). The response should appear similar to the following. If you have the network monitor visible in your browser, you will notice that the response code has changed to 404, indicating to the client system that the code entered does not exist.



Finally, experiment changing the URL to see how the Rhapsody configuration behaves.

Finally, experiment changing the URL to see how the Rhapsody configuration behaves.

JavaScript REST Client

Introduction

The JavaScript REST Client is configured in two parts.

- 1

The connection settings for the REST service you are using are defined using a REST Client in Rhapsody.

- 2

One or more JavaScript REST Client communication points are created that use the REST Client to connect to the service. JavaScript written in the communication point is used to define how the connection is configured and the message processed for each request. Where connection to a OAuth2 server is required for the purposes of authentication, a second REST Client can be associated with the communication point.

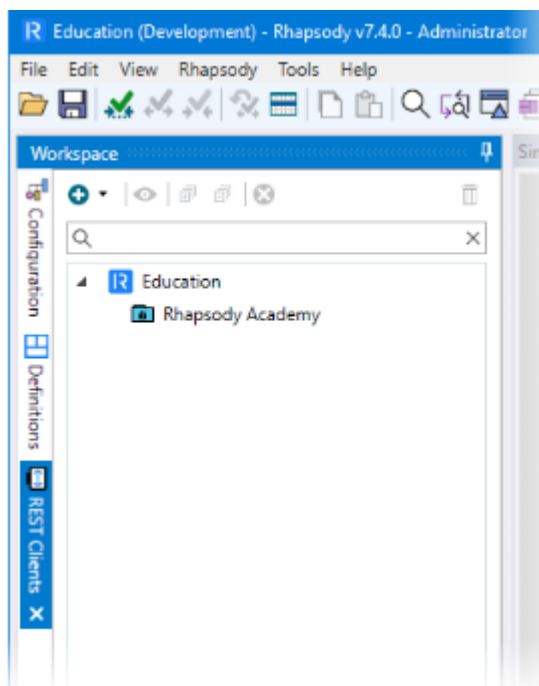
This approach allows a single REST Client to be defined once, then accessed by multiple communication points as required. This differs from the configuration approach used by the HTTP Client communication point, where each request method

must be configured with a separate communication point, and all shared settings must be defined using Rhapsody Environment Variables.

The scripted nature of the JavaScript REST Client allows significantly more configuration in the communication point than the HTTP Client communication point. This allows logic specific to the interface to be associated with the communication point. This helps keep any connected routes clean, making the solution easier to follow and understand.

Creating a New REST Client

REST Clients are created and edited from the **REST Clients** Workspace. This is accessed by selecting either the **View** menu or the **Managers** drop-down, then the **REST Clients** option.



You can add, edit and delete REST Clients from the Workspace panel. Adding and editing clients is done from the same properties screen. This screen has three tabs, **General**, **Proxy**, and **Security**.

REST Client Properties

General Proxy Security

Proxy Settings

☐ Proxy Enabled

Proxy Hostname:

Proxy Port:

Proxy Username:

Proxy Password:

☐ Use variable for proxy password

Help OK Cancel

REST Client Properties

General Proxy Security

HTTP(S) Settings

Protocol:

☐ Disable hostname verification

Authentication:

Username: *

Password: *

☐ Use variable for password

SSL Protocol:

SSL Cipher Suites:

Trusted Certificates: ...

Client Certificates: ...

Help OK Cancel

The screenshot shows the 'REST Client Properties' dialog box with the 'General' tab active. The 'REST Client Information' section contains fields for 'Client Name' (filled with 'REST Client'), 'Hostname', 'Port' (filled with '443'), 'Relative URL' (filled with '/'), 'Description', and 'Connection Timeout (ms)' (filled with '10000'). The 'Default Settings' section includes 'Content Type', 'Headers' (filled with '0 headers'), and 'Default Response Handling' (with dropdowns for '4xx Error' set to 'Treat as message error' and '5xx Error' set to 'Treat as connection error'). The 'OAuth2' section has an 'Access Token Cache Size' field filled with '10'. At the bottom are 'Help', 'OK', and 'Cancel' buttons.

Items with an asterisk must be provided with a value. All other configuration options are optional, however, you should enter as much detail as possible as this makes configuration via a JavaScript REST Client simpler.

Once a REST Client has been configured, commit changes to the Rhapsody engine by performing a **Check In**.

JavaScript REST Client

The **JavaScript REST Client (v.2)** communication point links a configured **REST Client** with JavaScript to make requests to a REST service. The use of JavaScript in the communication point allows details of the interface to be configured in the communication point and not a connected route. This in turn makes any connected routes easier to follow.

The client may be configured in one of the following modes:

- Output
- Out->In

The **JavaScript REST Client (v.2)** has four configuration properties. Aside from the name of the **REST Client** being used to define the connection, all configuration needed to connect to a web service is defined in the **Request Javascript** property. Where the service you will be connecting to uses OAuth2 for authentication, the **OAuth2 REST**

Client property is used to specify a second REST Client. This is used to perform OAuth2 based authentication during the request.

The screenshot shows the 'JavaScript REST Client (v.2) Properties' dialog box with the 'Configuration' tab selected. The 'Connection Mode' is set to 'Out->In'. A note states: 'Properties with an asterisk (*) are required.' Below this is a table with the following properties:

Property	Value
REST Client *	FHIR Patients
OAuth2 REST Client	
Request Javascript *	/**Perform a GET request with the default options.*/var config = {};var response = client.get(config);
Read Timeout (ms) *	10000

Below the table is a large empty text area for additional configuration. At the bottom are 'Help', 'OK', and 'Cancel' buttons.

Configuring the JavaScript

On first opening the **Request Javascript** property, you will be confronted by a daunting quantity of comments. However, these comments simply embed a set of sample configurations. The JavaScript performs three tasks:

- Configure the request to be sent to the web service
- Perform the request
- Generate an output message (when running in Out->In mode)

To assist with performing these functions, a number of objects exist within the JavaScript for the communication point:

JavaScript Object	Description
input	A ROMessage object containing the message sent to the communication point. This is described in the Messaging Module of this course.
output	A message object that contains the output from the communication point. This can be written to the same way the output message is written to when using a JavaScript filter.
client	An object unique to this communication point. This object interfaces the JavaScript with the REST Client associated with the communication point and performs the required REST calls.
jwtBuilder	An object for building a JSON Web Token (JWT). This is needed if you are performing a OAuth2 Client Credentials Grant.
JWT	An object that contains a already built JWT. Methods exist on this object to retrieve details of the JWT.

Configure the Request to be Sent to the Web Service

Prior to calling the web service, a JavaScript object is used to create a configuration (for example, to request a specific URL, or content encoding) for the web service call. The set of possible configuration options is listed below:

EXIT COURSE

[REN-207 \[7.4\] Working with Web Services](#)

100% COMPLETE

1. Introduction
 1. [What are Web Services?](#)
2. REST Web Services
 1. [Understanding REST](#)
 2. [HTTP Client and Server](#)
 3. [Exercise - HTTP Client](#)
 4. [Exercise - HTTP Server](#)
 5. [JavaScript REST Client](#)
 6. [Exercise - JavaScript REST Client](#)

3. SOAP Web Services

1. [Understanding SOAP](#)
2. [SOAP Web Service Client](#)
3. [Exercise - SOAP Web Service Client](#)
4. [SOAP Web Service Hosting](#)
5. [Exercise - SOAP Web Service Hosting](#)

4. Testing Web Services

1. [Testing](#)
2. [End of Course](#)

[Lesson 5 - Exercise - HTTP Server](#)

Lesson content

Lesson 6 of 14

JavaScript REST Client

Introduction

The JavaScript REST Client is configured in two parts.

1. 1

The connection settings for the REST service you are using are defined using a REST Client in Rhapsody.

2. 2

One or more JavaScript REST Client communication points are created that use the REST Client to connect to the service. JavaScript written in the communication point is used to define how the connection is configured and the message processed for each request. Where connection to a OAuth2 server is required for the purposes of authentication, a second REST Client can be associated with the communication point.

This approach allows a single REST Client to be defined once, then accessed by multiple communication points as required. This differs from the configuration approach used by the HTTP Client communication point, where each request method must be configured with a separate communication point, and all shared settings must be defined using Rhapsody Environment Variables.

The scripted nature of the JavaScript REST Client allows significantly more configuration in the communication point than the HTTP Client communication point. This allows

logic specific to the interface to be associated with the communication point. This helps keep any connected routes clean, making the solution easier to follow and understand.

Creating a New REST Client

REST Clients are created and edited from the **REST Clients** Workspace. This is accessed by selecting either the **View** menu or the **Managers** drop-down, then the **REST Clients** option.

You can add, edit and delete REST Clients from the Workspace panel. Adding and editing clients is done from the same properties screen. This screen has three tabs, **General**, **Proxy**, and **Security**.

-
-
-

Items with an asterisk must be provided with a value. All other configuration options are optional, however, you should enter as much detail as possible as this makes configuration via a JavaScript REST Client simpler.

Once a REST Client has been configured, commit changes to the Rhapsody engine by performing a **Check In**.

JavaScript REST Client

The **JavaScript REST Client (v.2)** communication point links a configured **REST Client** with JavaScript to make requests to a REST service. The use of JavaScript in the communication point allows details of the interface to be configured in the communication point and not a connected route. This in turn makes any connected routes easier to follow.

The client may be configured in one of the following modes:

- Output
- Out->In

The **JavaScript REST Client (v.2)** has four configuration properties. Aside from the name of the **REST Client** being used to define the connection, all configuration needed to connect to a web service is defined in the **Request Javascript** property. Where the service you will be connecting to uses OAuth2 for authentication, the **OAuth2 REST Client** property is used to specify a second REST Client. This is used to perform OAuth2 based authentication during the request.

Configuring the JavaScript

On first opening the **Request Javascript** property, you will be confronted by a daunting quantity of comments. However, these comments simply embed a set of sample configurations. The JavaScript performs three tasks:

- Configure the request to be sent to the web service
- Perform the request
- Generate an output message (when running in Out->In mode)

To assist with performing these functions, a number of objects exist within the JavaScript for the communication point:

JavaScript Object	Description
input	A ROMessage object containing the message sent to the communication point. This is described in the Messaging Module of this course.
output	A message object that contains the output from the communication point. This is written to the same way the output message is written to when using a JavaScript for the communication point.
client	An object unique to this communication point. This object interfaces the JavaScript for the REST Client associated with the communication point and performs the request calls.
jwtBuilder	An object for building a JSON Web Token (JWT). This is needed if you are performing an OAuth2 Client Credentials Grant.
JWT	An object that contains a already built JWT. Methods exist on this object to retrieve the JWT.

Configure the Request to be Sent to the Web Service

Prior to calling the web service, a JavaScript object is used to create a configuration (for example, to request a specific URL, or content encoding) for the web service call. The set of possible configuration options is listed below:

Configuration Option	Description
accept-type	Defines the Accept request-header field. This can be used to specify certain media types which are acceptable for the response. Example: 'application/xml+fhir'
relative-url	Defines the URL path to use relative to the URL defined within the REST Client configured on the communication point. Example: '/patient/1'
headers	Defines the request headers to be included in the request sent. These should be defined in an array as shown in the example here. Multiple header values can be defined by delimiting the values with commas. Example: [{'header':'headerA','value':'a'}, {'header':'headerB','value':'b,c,d'}]
parameters	Defines the request parameters to be included in the request sent. These should be defined in an array. Example: [{'parameter':'a','value':'a'}]
content-type	Defines the Content-Type request header field. This is used to declare the MIME type of the request body, and is mostly useful for PUT and POST requests. Example: 'application/xml'
enable-response-action	Defines if the configured response actions on the REST Client should be ignored. By default, this option is set to 'true' (response actions are enabled). Refer to Default Response Handling(opens in a new tab) for details. Example: 'true'

remove-headers	Defines headers from the REST Client configuration to remove. These are removed before extra headers are added via the headers option. Example: <pre>['header1', 'header2']</pre>
----------------	--

It is possible to use the message sent to the communication point by referring to the input JavaScript object (look at the relative-url and content-type changes below).

```
const config = {
  // Overrides the default accept-type header for this request
  'accept-type': 'text/xml',

  // Append property from input message to URL to be called
  'relative-url': '/referral/' + input.getProperty('EncounterId'),

  // Set a custom user agent for this request
  'headers': [
    {'header': 'User-Agent', 'value': 'Rhapsody REST Client'},
    {'header': 'X-Requested-With', 'value': 'Rhapsody'}
  ],

  // Set content type based on content in message
  'content-type': input.getProperty('MsgMimeType')
};
```

Perform the Request

A request is made to the web service by calling the method in the client object that matches the the desired HTTP verb. In all cases the method takes two optional parameters containing the request configuration and the message body.

Method	Description
destroy()	Performs an HTTP DELETE request and returns the response.
get()	Performs an HTTP GET request and returns the response.
head()	Performs an HTTP HEAD request and returns the response.
options()	Performs an HTTP OPTIONS request and returns the response.
post()	Performs an HTTP POST request and returns the response.
put()	Performs an HTTP PUT request and returns the response.

```
// Perform GET request
```

```
const response = client.get(config);
```

```
const messageBody = input.text;
```

```
// Perform POST request
```

```
const response = client.post(config, messageBody);
```

OAuth2

Where the web service requires OAuth2 authentication, a number of methods in the client object are used to make a request the OAuth2 server to obtain the access token.

Method	Description
requestAccessToken()	Requests an access token from the OAuth2 server. The method requires a configuration object (similar to the object used for other web service calls) and may optionally also include form parameters and a JWT as required.
clearAccessTokenCache()	Deleted access tokens that are currently cached.

To help formatting authentication headers in your requests, Two helper function are defined.

Method	Description
basicAuth()	Generates a Basic Authentication header for HTTP requests.
bearerAuth()	Generates an HTTP Authorization header using the Bearer token scheme.

In the following example, we make a request to an OAuth2 server using the requestAccessToken method on the client object. The basicAuth helper function to set the username and password in the request header. The response is processed to obtain the token, which is the used added to the configuration for the main request using the bearerAuth method.

```
const oauthConfig = {
  // Use basic authentication for OAuth2 request. If using variables, these could also be
  // set directly in the
  // OAuth REST client.
  'headers': [
    {'header': 'Authorization', 'value': basicAuth(getVariable('CLIENT_ID'),
getVariable('CLIENT_SECRET'))}
  ]
}
```

```

}

// Make request to OAuth2 server for access token (grant_type is set for you to
client_credentials)

const accessTokenResponse = client.requestAccessToken(oauthConfig);

// Parse the response and get access token

const token = JSON.parse(accessTokenResponse.response).access_token

// Use Access Token

const config = {
  'relative-url': "/users",
  // Use the access token in the request
  'headers': [{ 'header': 'Authorization', 'value': bearerAuth(token) }]
};

const response = client.get(config);

```

Generates an Output Message

When the communication point is running in Out->In mode, the message passed back to the route is populated into an output JavaScript object using the same JavaScript properties as are available in the JavaScript filter. Normally, the output object will be populated using content from the response variable (generated when you perform the call to the web service).

The response variable will contains the following items:

Method	Description
headers	Returns an array of the HTTP response headers. Examples: response.headers[0] - Returns the name of the first response header.

Method	Description
	response.headers['Date'] - Returns the value of the 'Date' response header, or undefined if the header does not exist
response	Returns the body of the HTTP response as a Unicode string. Example: response.response
responseBytes	Returns the body of the HTTP response as a byte array. Useful when the response contains non-text data. Example: response.responseBytes
status()	Returns the HTTP status code returned by the server to the request. For example, 200 (if the request was successful) and 404 (if the URL path given cannot be found on the server). Example: response.status()

```
output.setProperty('statusCode', response.status());
```

```
output.setText(response.response, 'UTF8');
```

```
// Copying values from the response headers.
```

```
output.setProperty('First-Header-Name', response.headers[0]);
```

```
if (response.headers['Date']) {
    output.setProperty('Date-Header-Value', response.headers['Date']);
}
```

```
// Iterate over set of response headers
```

```
for (const h in response.headers) {
    const header = response.headers[h];
```

```
    output.setProperty('Header-' + header, response.headers[header]);  
}
```

Exercise - JavaScript REST Client

Introduction

In this exercise, we will connect to the same test REST web service as was used in the HTTP Client exercise. As before, the exercise will start by performing a simple get patient record request, then explore the other request methods. For reference, details of the web service can be found by browsing to <http://resources.edu.rhapsody.global/professional/services/REST/patient/> (opens in a new tab).

Create JavaScript REST Client

From the **View** menu, select the **REST Clients** option. From the resulting Workspace panel, select the locker where the REST Client will be created, then right-click and select **New REST Client**. Configure the new client as follows:

REST Client Information

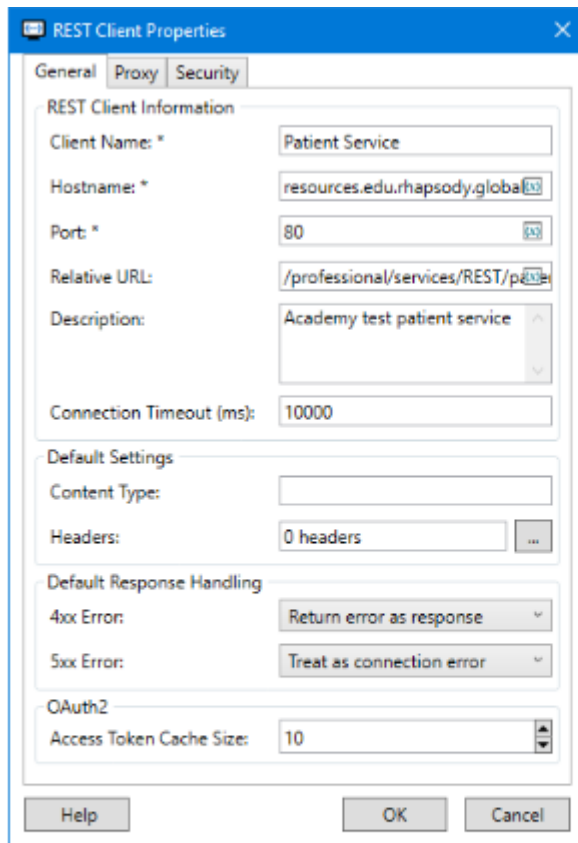
- **Client Name** - Patient Service
- **Hostname** - resources.edu.rhapsody.global
- **Port** - 80
- **Relative URL** - /professional/services/REST/patient/

Default Response Handling

- **4xx Error** - Return error as response

Security

- **Protocol** - HTTP



The various **REST Client Options** define how Rhapsody finds the web service. Under the **Default Response Handling** section, the **4xx Error** option should be changed to **Return error as a response** so that we can inspect details associated with any errors from the web server. Finally, select the **Security** tab and make sure the protocol is set to **HTTP**, this tells Rhapsody to connect to the web service over an insecure HTTP connection.

Create a JavaScript REST Client Communication Point

Create a new JavaScript REST Client (v.2) communication point. The communication point must be configured in Out->In mode.

Connection Mode: Out->In

Properties with an asterisk (*) are required.

Property	Value
REST Client *	Patient Service
OAuth2 REST Client	
Request Javascript *	// Append patient id we are looking for to base URL defined in REST Clientvar config = { 'relative-u
Read Timeout (ms) *	10000

From the **Configuration** tab, first select the **REST Client** you created above, then select the **Request Javascript** option and update the JavaScript to match the following:

```
// Append patient id we are looking for to base URL defined in REST Client
```

```
const config = {
  'relative-url': input.getProperty('PatientId')
};
```

```
// Perform GET request
```

```
const response = client.get(config);
```

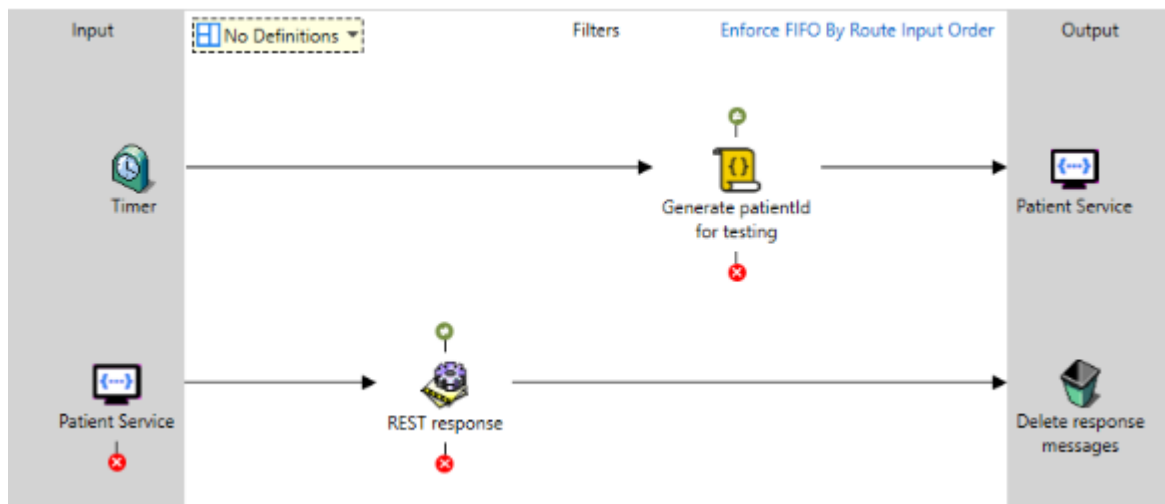
```
// Build output object with results from REST call
```

```
output.setProperty('statusCode', response.status);
```

```
output.setText(response.response, 'UTF8');
```

Create Route

Create a new route. Create and add a Timer communication point to the input of the route. This will be used to seed messages into the configuration. The JavaScript REST Client communication point should be added to both the input and output of the route. Finally, create a Sink communication point and add it to the output of the route. The Timer communication point should be linked to the JavaScript REST Client via a JavaScript (v.2) filter. Responses from the HTTP Client should be sent to a No-operation filter that then sends messages to the Sink communication point.



```
// Create the output message
```

```
const next = output.append(input[0]);
```

```
// Set of patient identifiers to test against the web service
```

```
const patientIdList = ['84568-4564', '84502-7664', '87619-9902', '84568-4341', '84568-3168', '91037-0197'];
```

```
// Patient id to search for. The incCounter() function is used to iterate over the set of patient identifiers.
```

```
const patientId = patientIdList[incCounter('Web Service Testing') % 6];
```

```
// Define message property to hold the patient to be searched
```

```
next.setProperty('PatientId', patientId);
```

Testing the Web Service

Check in, then start the configuration you have just created. Open the Management Console and inspect a range of messages output from the JavaScript REST Client communication point you created. Select the **Response received by message tracking** option to view the response associated with each message sent.

Extension - Modify Communication Point to Support All Request Types

The JavaScript REST Client communication point can be modified to process all the request types that can be sent to the web service. This is different to the HTTP Client communication point, where each instance is only able to process one type of request.

JavaScript REST Client

To do this, change the JavaScript associated with the JavaScript REST Client communication point to the following:

```
const action = input.getProperty('Action');

let config = {};

if (action !== 'Add') {

    // Concatenate base URL with patient id for request

    config = {

        'relative-url': input.getProperty('PatientId')

    };

}

// Choose appropriate request method based on the type of action being performed

let response;

switch (action) {

    case ('Get'):

        response = client.get(config);

        break;

    case ('Delete'):

        response = client.destroy(config);
```

```

        break;
    case ('Add'):
    case ('Update'):
        // Add and Update operations are performed with the same method
        // Include message body in request
        response = client.put(config, input.getText('UTF8'));
        break;
}

```

```

// Build output object with results from REST call
output.setProperty('statusCode', response.status);
output.setText(response.response, 'UTF8');

```

The revised JavaScript requires that a message property called Action is set on the message indicating how it should be processed.

Route Changes

Modify your route to set an appropriate action for each message. Update the JavaScript filter as follows:

```

// Create the output message
const next = output.append(input[0]);

// Set of patient identifiers to test against the web service
const patientIdList = ['84568-4564', '84502-7664', '87619-9902', '84568-4341', '84568-3168', '91037-0197'];

// Patient id to search for. The incCounter() function is used to iterate over the set of patient identifiers.
const patientId = patientIdList[incCounter('Web Service Testing') % 6];

```

```
// Set of valid actions.
```

```
const actionList = ['Get', 'Add', 'Update', 'Delete'];
```

```
// Action to perform.
```

```
const action = actionList[incCounter('Web Service Testing Action') % 4];
```

```
// Define message property to hold the patient to be searched and action to perform.
```

```
next.setProperty('PatientId', patientId);
```

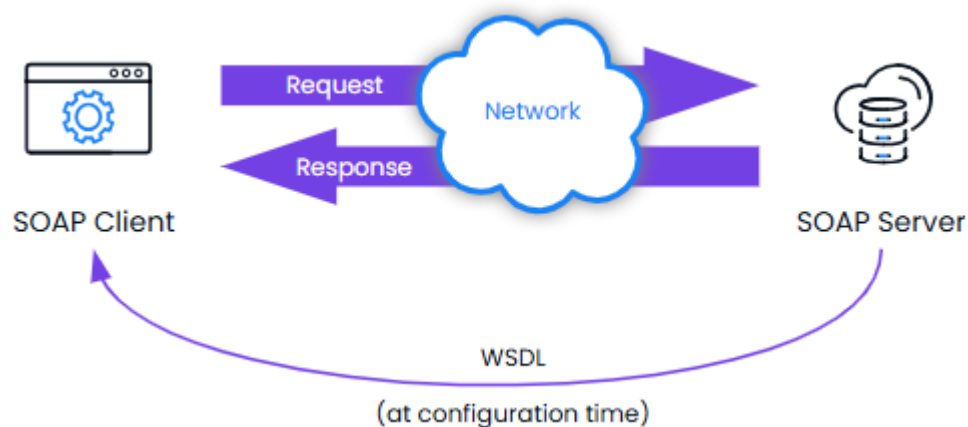
```
next.setProperty('Action', action);
```

Remember to add some valid XML to the Rhapsody message body when either the add or update actions are set. Refer back to the Add and Update extension in the HTTP Client exercise for an approach and test file that could be used.

Understanding SOAP

Introduction

SOAP (Simple Object Access Protocol) is an XML-based protocol for managing web service communications. The protocol defines operations on the SOAP Server which, when called via a request, perform some tasks and return the result via a response. The protocol also makes use of WSDL files to provide client systems with the necessary details to be able to connect to the server and perform a request.



The request and response messages are defined as XML and contain the following parts:

- An envelope identifying the message as a SOAP message.
- A SOAP header containing metadata relating to the request (such as a message id).
- A Body containing the request / response.
- Optionally a Fault section encapsulated in the body to report errors.

The SOAP Envelope provides the framework for the message and references the SOAP Envelope namespace as well as the SOAP Encoding namespace, which is used to define the data types used in the message.

Further Reading

- <http://en.wikipedia.org/wiki/SOAP>(opens in a new tab)
- <https://msdn.microsoft.com/en-us/library/ms996507.aspx>(opens in a new tab)

Namespaces

XML Namespaces are used extensively within SOAP messaging. Understanding them is important to getting the best out of SOAP messaging.

What is an XML Namespace?

A namespace is a way of grouping related components in an XML message together. This might be required for complex documents or where an XML document is made up from nodes that originate in a number of different specifications. This is the case for SOAP messages, where the envelope and header are part of the SOAP specification, while the body is unique to the interface. (The body could also require nodes from other specifications to populate either the request or response).

For example, imagine that we had the following XML document containing both patient and clinic names:

```
<appointment>
  <booking>
    <date>2014-11-18T14:30:00</date>
    <clinic>
      <name>Gastroenterology</name>
      <location>
```

```

    <code>1E-B19</code>

    <description>Reception E, Level 1, Outpatient Building</description>

  </location>

</clinic>

<status>Confirmed</status>

</booking>

<patient>

  <identifier>AAQ9954</identifier>

  <sex>M</sex>

  <name>

    <given>Fred</given>

    <family>Dagg</family>

  </name>

</patient>

</appointment>

```

If we were given one of the two name nodes in the example to process without any additional information, we would not know if we had been given a patient name or a clinic name. The naive solution to this problem is alter all node names to make sure they have a globally unique meaning. However, as an XML document increases in size and complexity or when the document relies on nodes for different specifications, this quickly becomes unworkable.

Prefixes

The first step on the path to namespaces is to add a prefix to the start of every node. The prefixes will group the various nodes by specification or origin. This is done by adding a prefix tag followed by a colon to the node name. In the example below, two prefixes have been created: appt and pat.

```

<appt:appointment>

  <appt:booking>

    <appt:date>2014-11-18T14:30:00</appt:date>

```

```

<appt:clinic>
  <appt:name>Gastroenterology</appt:name>
  <appt:location>
    <appt:code>1E-B19</appt:code>
    <appt:description>Reception E, Level 1, Outpatient Building</appt:description>
  </appt:location>
</appt:clinic>
<appt:status>Confirmed</appt:status>
</appt:booking>
<pat:patient>
  <pat:identifier>AAQ9954</pat:identifier>
  <pat:sex>M</pat:sex>
  <pat:name>
    <pat:given>Fred</pat:given>
    <pat:family>Dagg</pat:family>
  </pat:name>
</pat:patient>
</appt:appointment>

```

With the addition of node prefixes, it is possible to distinguish the pat:name (patient name) node from the appt:name (clinic name) node. However, on their own, a prefix says very little about the data or its origin.

Namespaces

To solve this, an XML namespace includes a declaration that uses a URI (Uniform Resource Identifier) to identify the owner and nature of the content. In the revised example, these declarations appear on the first line of the XML.

```

<appt:appointment
xmlns:appt="http://academy.rhapsody.health/2020/Rhapsody/Appointment"
  xmlns:pat="http://academy.rhapsody.health/2020/Rhapsody/Patient">

```

```

<appt:booking>
  <appt:date>2014-11-18T14:30:00</appt:date>
  <appt:clinic>
    <appt:name>Gastroenterology</appt:name>
    <appt:location>
      <appt:code>1E-B19</appt:code>
      <appt:description>Reception E, Level 1, Outpatient Building</appt:description>
    </appt:location>
  </appt:clinic>
  <appt:status>Confirmed</appt:status>
</appt:booking>
<pat:patient>
  <pat:identifier>AAQ9954</pat:identifier>
  <pat:sex>M</pat:sex>
  <pat:name>
    <pat:given>Fred</pat:given>
    <pat:family>Dagg</pat:family>
  </pat:name>
</pat:patient>
</appt:appointment>

```

The namespace URI is normally specified as a URL, although it does not need to point to a valid Internet resource. In the above example, the domain name of the URL (academy.rhapsody.health) tells us the name of the organization who authored the namespace, while the path after the domain name (/2020/Rhapsody/Appointment and /2020/Rhapsody/Patient) describes the type of information associated with each namespace. The date helps us version the namespace, should any changes be required in future.

It is the URI that defines the namespace, not the prefix. The prefix should be thought of as a shorthand name or alias for the namespace URI. The declaration in the document performs this mapping. In the following two examples, the same namespaces are being

referred to with two different prefixes (appt & ns1, and pat & ns2) . From a content point of view, these documents are equivalent:

```
<appt:appointment
xmlns:appt="http://academy.rhapsody.health/2020/Rhapsody/Appointment"
    xmlns:pat="http://academy.rhapsody.health/2020/Rhapsody/Patient">
    <appt:booking>
        ...
    </appt:booking>
</appt:appointment>
```

```
<ns1:appointment
xmlns:ns1="http://academy.rhapsody.health/2020/Rhapsody/Appointment"
    xmlns:ns2="http://academy.rhapsody.health/2020/Rhapsody/Patient">
    <ns1:booking>
        ...
    </ns1:booking>
</ns1:appointment>
```

Namespaces and XPath Expressions

When inserting or extracting content from an XML message using an XPath expression, the prefix must be included as part of the expression. For example, to get a patient's family name in the example above, we would use the following XPath expressions:

```
/appt:appointment/pat:patient/pat:name/pat:family
```

If the prefix associated with a namespace is changed, the XPath expression will also need to change. To manage this, some configuration screens that deal with XML generated outside Rhapsody will define a namespace URI to prefix mapping. This allows XPath expressions to be written with a consistent set of prefixes.

XML Namespace Mapping		
	Prefix	Namespace URI
▶	ns1	http://academy.rhapsody.health/Rhapsody/Professional/Appointment
*		

Mapping the 'http://academy.rhapsody.health/Rhapsody/Professional/Appointment' namespace to the 'NS1' prefix

Be aware that, by default, Rhapsody will use the ns1 namespace prefix. You can change this where it makes the resulting XPath expressions easier to follow.

Further Reading

- [http://msdn.microsoft.com/en-us/library/aa468565.aspx\(opens in a new tab\)](http://msdn.microsoft.com/en-us/library/aa468565.aspx(opens in a new tab))
- http://en.wikipedia.org/wiki/Uniform_resource_identifier

• Anatomy of a Web Service Request

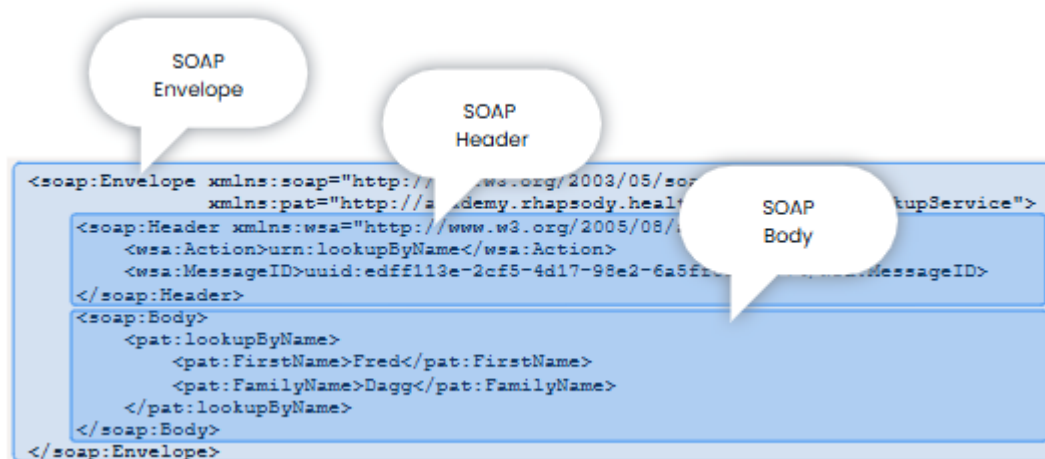
- In the following example, we look at the SOAP request against a lookupByName operation that when successful will return details associated with the search.

• Request

- The request sent to the server appears as follows:

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
  xmlns:pat="http://academy.rhapsody.health/2020/PatientLookupService">
  <soap:Header xmlns:wsa="http://www.w3.org/2005/08/addressing">
    <wsa:Action>urn:lookupByName</wsa:Action>
    <wsa:MessageID>uuid:edff113e-2cf5-4d17-98e2-6a5ff09ce377</wsa:MessageID>
  </soap:Header>
  <soap:Body>
    <pat:lookupByName>
      <pat:FirstName>Fred</pat:FirstName>
      <pat:FamilyName>Dagg</pat:FamilyName>
    </pat:lookupByName>
  </soap:Body>
</soap:Envelope>
```

The complete message is referred to as the envelope. The header contains a WS-Addressing action and message Id field, while the body contains the details of the request being performed. The operation that will be performed by this request is lookupByName. Two parameters FirstName and FamilyName have been included in the request. These parts are summarized in the illustration below.



If this request was configured in Rhapsody, the communication points could be configured to automatically generate and populate the envelope and header with the correct details. Only the content of the body might need to be configured.

Response

The response to this request might be as follows:

```
<soapenv:Envelope xmlns:soapenv="http://www.w3.org/2003/05/soap-envelope">
  <soapenv:Header xmlns:wsa="http://www.w3.org/2005/08/addressing">
    <wsa:RelatesTo>uuid:edff113e-2cf5-4d17-98e2-6a5ff09ce377</wsa:RelatesTo>
  </soapenv:Header>
  <soapenv:Body>
    <pat:lookupByName
      xmlns:pat="http://academy.rhapsody.health/2020/PatientLookupService">
      <pat:PatientId>AAQ9954</pat:PatientId>
      <pat:FirstName>Fred</pat:FirstName>
      <pat:FamilyName>Dagg</pat:FamilyName>
      <pat:Sex>M</pat:Sex>
```

```
<pat:DateOfBirth>1948-07-29</pat:DateOfBirth>

</pat:lookupByName>

</soapenv:Body>

</soapenv:Envelope>
```

The structure of the body in the response is specific to each web service and the nature of the data being returned.

Error

If an error occurs when processing the request, the web service host would return an error that indicated the cause of the fault.

```
<soapenv:Envelope xmlns:soapenv="http://www.w3.org/2003/05/soap-envelope">

  <soapenv:Body>

    <soapenv:Fault>

      <soapenv:Code>

        <soapenv:Value>soapenv:Receiver</soapenv:Value>

      </soapenv:Code>

      <soapenv:Reason>

        <soapenv:Text xml:lang="en-US">

          The required parameter 'FamilyName' is missing.

        </soapenv:Text>

      </soapenv:Reason>

    </soapenv:Fault>

  </soapenv:Body>

</soapenv:Envelope>
```

Where Rhapsody is hosting a SOAP web service, it is able to detect and automatically respond to many error conditions. Both Client and Hosting communication points contains options for dealing with error messages.

WSDL Files

The configuration of a SOAP web service will normally be described by a WSDL (Web Services Description Language) file. The file contains connection details for the web service along with information on the accepted functions that can be called. The file is an XML file that is normally generated by the SOAP service.

Developers are not expected to edit the file directly, since most systems that provide a SOAP server function (including Rhapsody) are able to automatically generate the WSDL file from the service definition.

Most SOAP clients are able to process the file to auto-configure the web service connection details. This significantly simplifies the process of configuring a SOAP client system.

Further Reading

- https://en.wikipedia.org/wiki/Web_Services_Description_Language(opens in a new tab)
- <http://www.w3.org/TR/wsdl>

SOAP Web Service Client

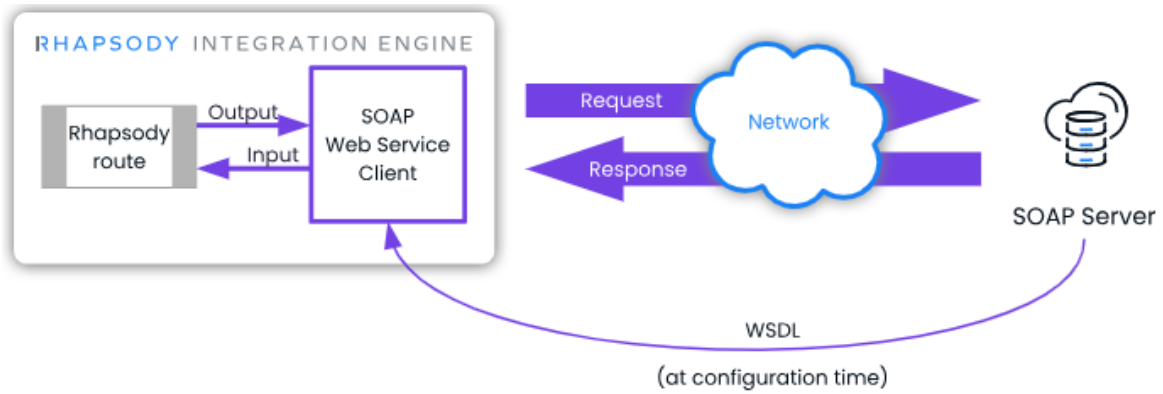
overview

The SOAP Web Service Client connects to an external web service host using a WSDL file to set up connection details. The following communication modes are supported:

- Output
- Out->In

The communication point takes a message and uses it along with the message properties to construct an XML SOAP request. The communication point will frequently use a template to construct the SOAP message by extracting fields and properties from the message. When processing the response message, if the message body is not updated, the body sent to the communication point will be used to construct the new message. This allows one or more message properties to be updated by a web service without the message body being updated.

The diagram below illustrates how the SOAP Web Service Client communication point in Rhapsody interacts with an external system over SOAP. The external system could be any system that understands SOAP, including another Rhapsody server.

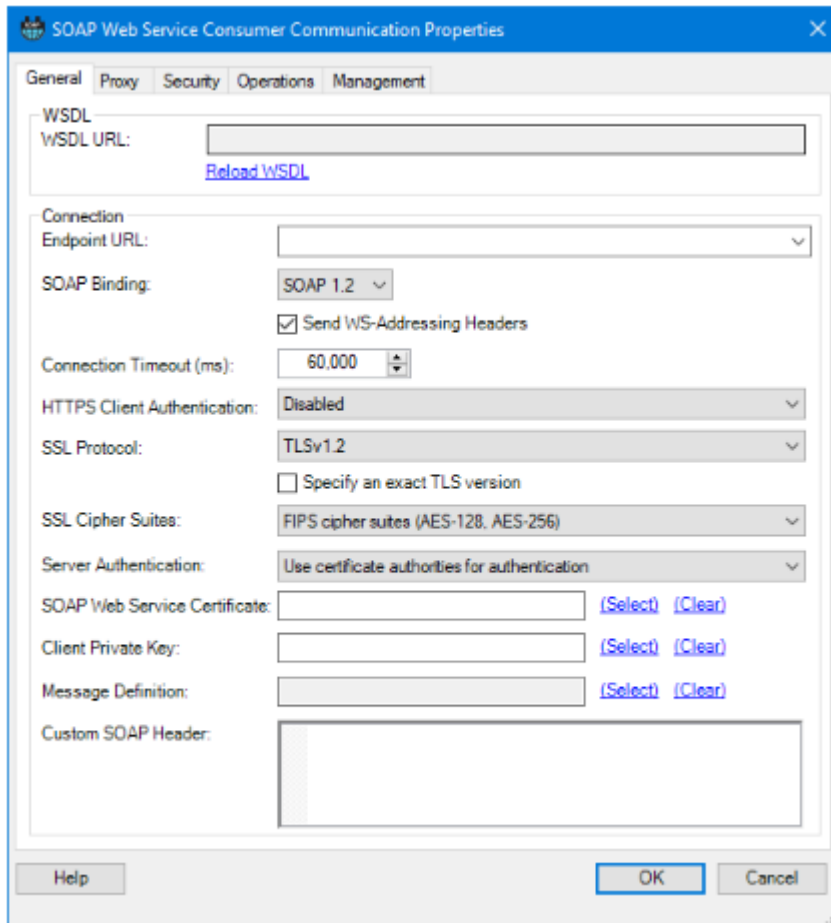


When the communication point is configured, a WSDL file can be downloaded from a URL on the server. When the communication point is running, messages sent to the communication point are translated into a request sent to the external system. Where the external system sends a response message back to Rhapsody, this is translated into the route input.

Configuration

The Configuration tab for the SOAP Web Service Client shows a summary of the configuration. To configure the communications point, select **Edit Communications Point Configuration**. This will display the SOAP Web Service Consumer Communications Properties screen.

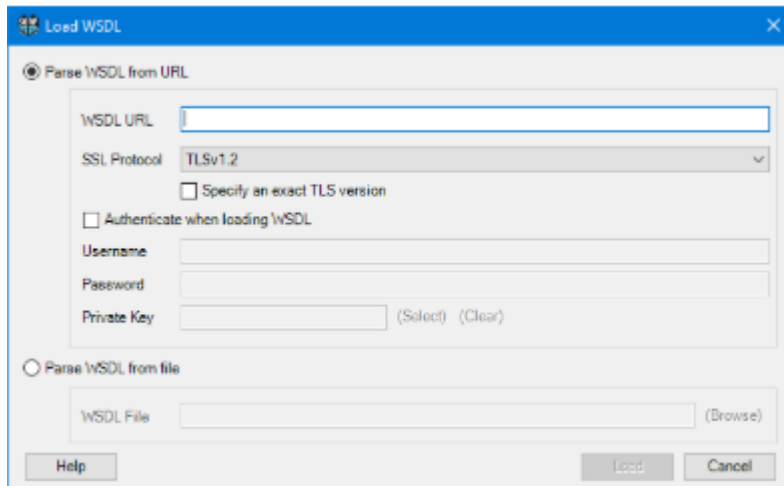
General



The image shows a Windows-style dialog box titled "SOAP Web Service Consumer Communication Properties". It has a blue title bar with a close button (X) in the top right corner. Below the title bar is a tabbed interface with five tabs: "General", "Proxy", "Security", "Operations", and "Management". The "General" tab is currently selected. The "WSDL" section at the top contains a "WSDL URL:" text box and a blue link labeled "Reload WSDL". Below this is the "Connection" section, which includes several configuration options: "Endpoint URL:" with a dropdown arrow, "SOAP Binding:" set to "SOAP 1.2" with a dropdown arrow, a checked checkbox for "Send WS-Addressing Headers", "Connection Timeout (ms):" set to "60,000" with a spinner, "HTTPS Client Authentication:" set to "Disabled" with a dropdown arrow, "SSL Protocol:" set to "TLSv1.2" with a dropdown arrow, an unchecked checkbox for "Specify an exact TLS version", "SSL Cipher Suites:" set to "FIPS cipher suites (AES-128, AES-256)" with a dropdown arrow, "Server Authentication:" set to "Use certificate authorities for authentication" with a dropdown arrow, "SOAP Web Service Certificate:" with a text box and "(Select) (Clear)" links, "Client Private Key:" with a text box and "(Select) (Clear)" links, "Message Definition:" with a text box and "(Select) (Clear)" links, and "Custom SOAP Header:" with a large text area. At the bottom of the dialog are three buttons: "Help", "OK", and "Cancel".

The first step when configuring this communication point is to select the **Reload WSDL** link to display the **Load WSDL** screen.

From this screen, a WSDL file can either be loaded from a URL or (if the web service host is unable to serve the file directly) a file. Selecting **Load** will cause Rhapsody to load and parse the WSDL file, which auto-configures most of the options under the **General** tab. In the unlikely event that the web service does not provide a WSDL file, all the options for the web service will need to be manually entered based on information provided about the web service.

The image shows a 'Load WSDL' dialog box with a blue title bar. It contains two radio buttons: 'Parse WSDL from URL' (selected) and 'Parse WSDL from file'. The 'Parse WSDL from URL' section includes a 'WSDL URL' text field, an 'SSL Protocol' dropdown menu set to 'TLSv1.2', a 'Specify an exact TLS version' checkbox, an 'Authenticate when loading WSDL' checkbox, and fields for 'Username', 'Password', and 'Private Key' (with 'Select' and 'Clear' buttons). The 'Parse WSDL from file' section includes a 'WSDL File' text field and a '(Browse)' button. At the bottom are 'Help', 'Load', and 'Cancel' buttons.

Where it is possible for the **Endpoint URL** to change between environments, a Rhapsody Variable can be used for all or part of this field.

When configuring the communication point, it is possible to specify a **Message Definition** for the messages output to the communication point from a route. This allows fields in an HL7 or similar message to be referenced by the communication point and inserted directly into the SOAP request.

Operations

The Operations tab allows the user to specify the web service actions performed by this communications point. Select **Add Operation** to add a new operation, or Edit Operation to modify an existing one. Either action displays the following screen:

Operation Editor

Operation

Name: Namespace:

Mode: SOAP Action:

Request WS-Addressing Action: ☐ Enable MTOM

Request Response Fault

Configure how the outgoing SOAP request is built from the Rhapsody message.

Mode:

Template

```

1 <pat:lookupByName xmlns:pat="http://edu.rhapsody.global/patient/PatientLookupService"
2   <!--Zero or more repetitions!-->
3   <pat:FirstName>string</pat:FirstName>
4   <!--Optional!-->
5   <pat:FamilyName>string</pat:FamilyName>
6 </pat:lookupByName>

```

Content Insertion

	Source Type	Property / Field Name	...	Content Type	Target XPath
*					

Signing / Encryption

Edit Signing and Encryption Parameters

The signing and encryption options are only available when using the asymmetric or symmetric security bindings.

Help OK Cancel

From the **Name** field drop-down list select the desired action to perform, this list is provided by the web service host via the WSDL. On selection of an action, Rhapsody will auto-configure most of the other fields on the screen.

The **Request** tab determines how the SOAP message to be sent to the web service host is constructed. The **Response** tab determines how the SOAP response received from the host is processed, and the Fault tab defines how error messages from the host are treated. The **Mode** drop-down list on each tab selects the method for processing the selected message type.

For all tabs, there is the option to perform some action using JavaScript. When that option is selected, a default JavaScript is displayed that either performs a basic action (for the Request) or contains a comment that details the actions that can be performed. Where JavaScript is used, it should only be used to preprocess the message for the communication point and not used to modify the content of the message.

Care should be taken sending or receiving a message body via a SOAP web service, as some special characters (including the HL7 segment separator) may be modified or corrupted.

Request

The request can be constructed using a Template, the Message or JavaScript to build the outgoing message. When a template or the message is being used to create the outgoing SOAP request, the Content Insertion section is used to insert message properties and additional content into the message.

Response

The **Response** tab is only available when the operation's **Mode** is *Request-Response*. The Response can be processed using the entire response as the message, JavaScript, or extract the message and properties from the response.

When extracting the message and properties from the response, the **XML Namespace Mapping** and **Content Extraction** fields are used to extract information from the SOAP response using XPath expressions.

If the response is configured to not update the message body, then the message body originally sent to the communication point will be output. This effectively enables one or more message properties to be updated by the web service call without altering the message body.

When doing content extraction from the SOAP message, remember to take note of the prefixes that have been defined for the XML namespaces that are associated with the SOAP definition.

Fault

Any response message containing a SOAP fault message will be processed in accordance with the settings on this tab. Rhapsody can be configured to execute some JavaScript, return the entire message to the route, or raise an error against the communications point.

Exercise - SOAP Web Service Client

Introduction

For this exercise, we will be connecting to a test SOAP Web Service containing a few patient records. The WSDL for this web service can be found here: <http://resources.edu.rhapsody.global/professional/services/SOAP/Patients?wsdl> ([opens in a new tab](#)). The web service contains four operations:

Operation	Description
AddRecord	Adds a new patient to the service.
DeleteRecord	Deletes an existing patient record from the service.
GetRecord	Gets an existing record from the service.
UpdateRecord	Updates an existing record in the patient service.

This Web Service contains details for the same set of patients used previously with the REST clients.

In this exercise, we will be using the SOAP Web Service Client to connect to and test the web service.

Create Communication Point

Create a new SOAP Web Service Client communication point. The communication point should have mode of Out->In. From the **Configuration** tab, select **Edit Communication Point Configuration**.

Select the **Reload WSDL** link and enter a **WSDL**

URL of <http://resources.edu.rhapsody.global/professional/services/SOAP/Patients?wsdl> *(opens in a new tab)*. To load the contents of the WSDL into the communication point, select **Load**. Rhapsody will contact the server to download and parse the WSDL. Once Rhapsody has confirmed that the WSDL is valid, it will present a success message. Select **Close** on this confirmation dialog box.

Inspect the settings on the **General** tab. These will have all been updated as a result of loading the WSDL.

Select the **Operations** Tab. Select **Add Operation**. This will open the Operation Editor.

Choose the **Name** drop-down and select the GetRecord operation. Notice how the contents of the operation editor changes when the operation being performed is changed.

Operation Editor

Operation

Name: Namespace:

Mode: SOAP Action:

Request WS-Addressing Action: ☐ Enable MTOM

Request Response Fault

Configure how the outgoing SOAP request is built from the Rhapsody message

Mode:

Template

```

1 <pat:GetRecord xmlns:pat="http://edu.rhapsody.global/professional/patient">
2   <pat:Identifier>string</pat:Identifier>
3 </pat:GetRecord>

```

Content Insertion

	Source Type	Property / Field Name	...	Content Type	Target XPath
*	Property	PatientId		Text	/pat:GetRecord/pat:Identifier

Signing / Encryption

Edit Signing and Encryption Parameters

The signing and encryption options are only available when using the asymmetric or symmetric security bindings.

Help OK Cancel

Under the **Content Insertion** section, add a new row into the table. This will add a message property called PatientId into the message sent to the SOAP service.

Field	Value
Source Type	Property
Property / Field Name	PatientId
Content Type	Text
Target XPath	/pat:GetRecord/pat:Identifier

Select the **Response** tab and change the **Mode** to **Use entire SOAP envelope as Rhapsody message body**. This returns the whole message to the route without any processing in the communication point. This allows us to see what is being returned from the Web Service.

Operation Editor

Operation
 Name: Namespace:
 Mode: SOAP Action:
 Request WS-Addressing Action: ☐ Enable MTOM

Request Response Fault

Configure how the incoming SOAP response is used to populate the Rhapsody message.

Mode:

XML Namespace Mapping

Prefix	Namespace URI
ns1	http://edu.rhapsody.global/professional/patient
*	

Content Extraction

Source XPath	Target Type	Target Property / Field
*		

Signing / Encryption

Edit Signing and Encryption Parameters

The signing and encryption options are only available when using the asymmetric or symmetric security bindings.

Help OK Cancel

Select the **Fault** tab and change the **Mode** to **Return Fault To Route**. This lets us see the content of any errors returned from the Web Service.

Operation Editor

Operation
 Name: Namespace:
 Mode: SOAP Action:
 Request WS-Addressing Action: ☐ Enable MTOM

Request Response Fault

Configure how incoming SOAP faults are handled by the communication point.

Mode:

JavaScript

```

1 // Check the fault code
2 //if (fault.faultCode.localName == 'Receiver') {
3 // Treat as a connection error
4 //fault.treatAsConnectionError();
5 //}
6 // Check fault subcode (soap 1.2 only)
7 //if (fault.SubCodes.length != 0 && fault.SubCodes[0].localName == 'FailedAuth
8 // Treat as message error
9 //fault.treatAsMessageError('Security error');
10 //}
11 // Insert fault into message body (assumed to be XML here)
12 //default xml namespace = 'http://www.orionhealth.com/2010/01/rhapsody';
13 //var msg = input.xml;
14 //msg..result.appendChild(fault.envelope..soapNs::Fault);
15 //output.text = msg;
  
```

Signing / Encryption

Edit Signing and Encryption Parameters

The signing and encryption options are only available when using the asymmetric or symmetric security bindings.

Help OK Cancel

Create Route

Create a new Route.

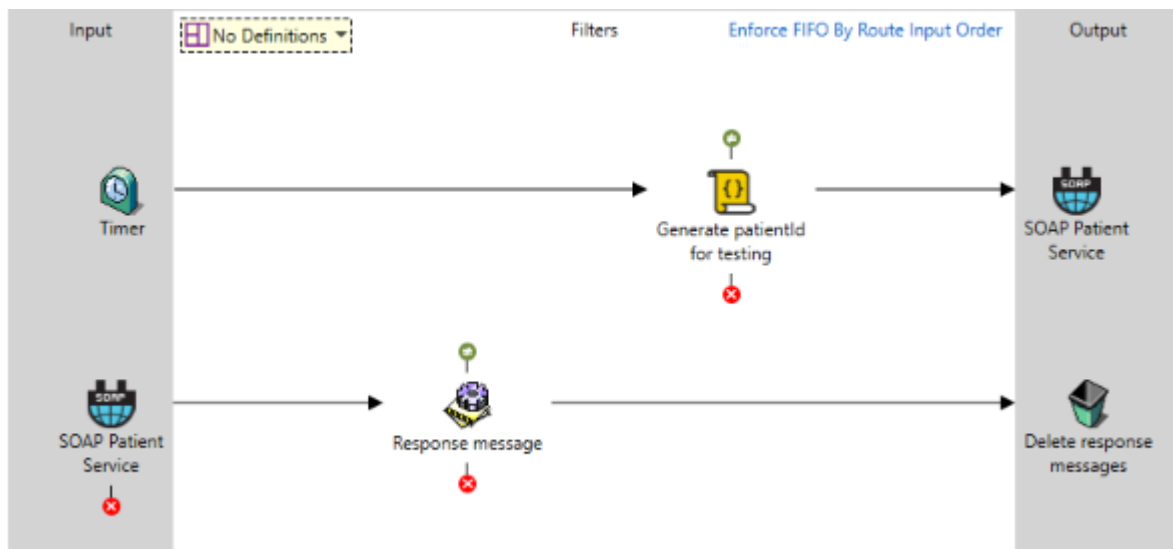
Create and add a Timer communication point to the input of the route. This will be used to seed messages into the configuration.

The SOAP Web Service Client communication point should be added to both the input and output of the route.

Finally, create a Sink communication point and add it to the output of the route.

The Timer communication point should be linked to the Web Service Client via a JavaScript (v.2) filter.

Responses from the HTTP Client should be sent to a No-operation filter that then sends messages to the Sink communication point.



JavaScript

The JavaScript filter should be configured as follows:

```
// Create the output message
```

```
const next = output.append(input[0]);
```

```
// Set of patient identifiers to test against the web service
```

```
const patientIdList = ['84568-4564', '84502-7664', '87619-9902', '84568-4341', '84568-3168', '91037-0197'];
```

```
// Patient id to search for. The incCounter() function is used to iterate
```

```
// over the set of patient identifiers.  
  
const patientId = patientIdList[incCounter('Web Service Testing') % 6];  
  
// Define message property to hold the patient to be searched  
next.setProperty('PatientId', patientId);
```

Testing The Web Service

Check in, then start the configuration you have just created.

Open the Management Console and inspect a range of messages output from the SOAP Web Service Client communication point you created.

Select the **Response received by message tracking** option to view the response associated with each message sent.

Message View 1.26697519299712.472

Displaying message 2 of 11 from the result set named 'Patients Web Service Output Archive' [Back to Result Set](#)

[Reinject](#) [Redirect Copy](#) [Redirect Edited Copy](#) [Resend](#)

Message Events

[Always Show Outputs](#) Last refreshed at 11:47

COMPONENT	TIMESTAMP	DESCRIPTION
Generate test messages	Aug 2, 2021 11:47:30.502	Received by communication point
Test SOAP Service	Aug 2, 2021 11:47:30.502	Placed on route
Generate PatientId for testing	Aug 2, 2021 11:47:30.502	Filtering succeeded; took 1.059 ms
Patients Web Service	Aug 2, 2021 11:47:30.502	Delivered to communication point
	Aug 2, 2021 11:47:30.676	Sent by communication point, total pro...
	Aug 2, 2021 11:47:30.681	Response received by message tracking

Message Properties

Output Message

Property Name	Property Value
InputTime	1627861650502
PatientId	87619-9902
rhapsody:InputCommunicationPoint	Generate test messages
rhapsody:InputCommunicationPoin...	6656

Showing 4 of 4

Response Message

[View](#)

Property Name	Property Value
InputTime	1627861650681
PatientId	87619-9902
rhapsody:BodyEncoding	UTF-8
rhapsody:InputCommunicationPoint	Generate test messages
rhapsody:InputCommunicationPoin...	6656
rhapsody:ResponseLatency	172

Showing 6 of 6

Message Body

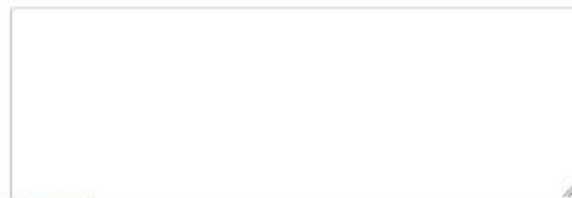
Tree View

Flat Wire

Output Message

Show Whitespace Encoding: windows-1252

Size: 0 bytes



[Download](#)

Response Message

[View](#)

XML Encoding: UTF-8

Size: 490 bytes

```
<ns1:GetRecordResponse>
  <ns1:Identifier>
    87619-9902
  </ns1:Identifier>
  <ns1:GivenName>
    Lila
  </ns1:GivenName>
  <ns1:GivenName>
    Betty
  </ns1:GivenName>
</ns1:GetRecordResponse>
```

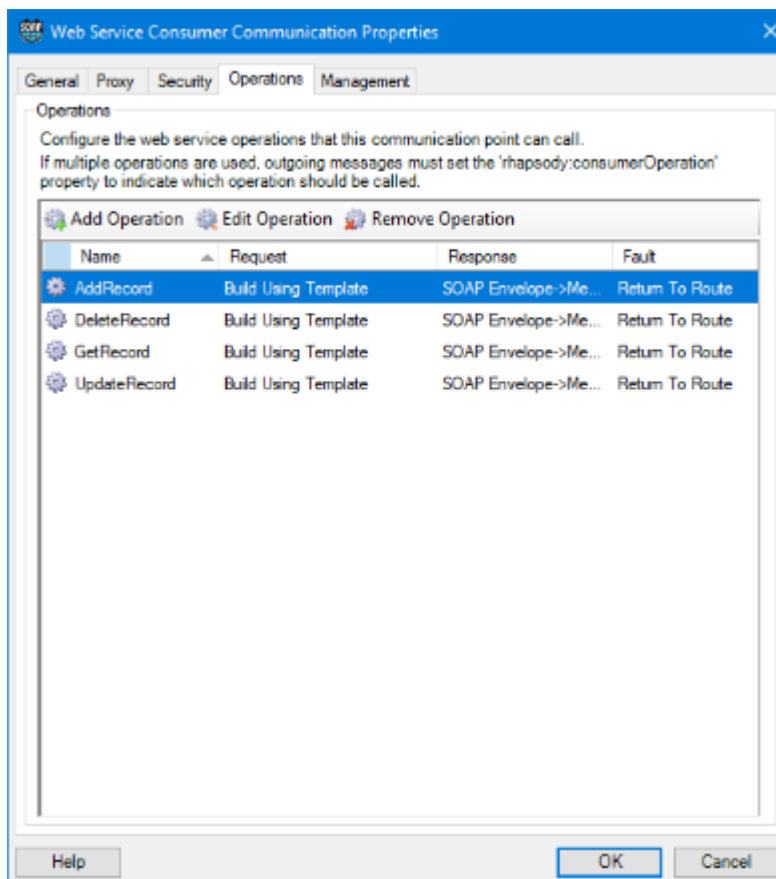
[Download](#)

Extension - Modify Communication Point to Support All Operations

The SOAP Web Service Client communication point can be configured to process all the request types that can be sent to the Web Service. To do this, add the DeleteRecord, UpdateRecord and AddRecord operations with the same configuration as the previously created GetRecord operation (you don't need to add the PatientId message property to the AddRecord operation). The correct XPath expressions differs for each operation. The table below lists the correct expressions to use.

Operation	XPath expression required
AddRecord	<i>not required</i>
DeleteRecord	/pat:DeleteRecord/pat:Identifier
UpdateRecord	/pat:UpdateRecord/pat:Identifier

Once all operations have been added, the **Operations** tab of the Web Service Client communication point should appear as follows:



Next, update the JavaScript on the route to the following:

```
// Create the output message
```

```
const next = output.append(input[0]);
```

```
// Set of patient identifiers to test against the web service
```

```
const patientIdList = ['84568-4564', '84502-7664', '87619-9902', '84568-4341', '84568-3168', '91037-0197'];
```

```
// Patient id to search for. The incCounter() function is used to iterate  
// over the set of patient identifiers.
```

```
const patientId = patientIdList[incCounter('Web Service Testing') % 6];
```

```
// Define message property to hold the patient to be searched  
next.setProperty('PatientId', patientId);
```

```
// Set of actions we can perform against the test SOAP Server  
const actionList = ['GetRecord', 'UpdateRecord', 'AddRecord', 'DeleteRecord'];
```

```
// Set action to perform.  
const action = actionList[incCounter('Web Service Testing Action') % 4];
```

```
// Define message property to hold the action to be performed  
next.setProperty('rhapsody:consumerOperation', action);
```

Check in all changes and start the configuration. Open the Management Console and review the response for each message sent to the Web Service.

Message Properties ▾

Output Message

Property Name	Property Value
InputTime	1563373734840
PatientId	91037-0197
rhapsody:consumerOperation	AddRecord
rhapsody:InputCommunicati...	Generate test messages
rhapsody:InputCommunicati...	221

Showing 5 of 5

Response Message

[View](#)

Property Name	Property Value
InputTime	1563373735159
PatientId	91037-0197
rhapsody:BodyEncoding	UTF-8
rhapsody:consumerOperation	AddRecord
rhapsody:InputCommunicati...	Generate test messages
rhapsody:InputCommunicati...	221
rhapsody:ResponseLatency	301

Showing 7 of 7

Message Body ▾

Tree View ▸

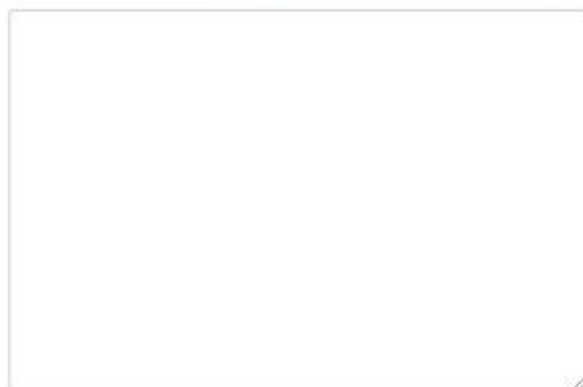
Flat Wire ▾

Output Message

Show Whitespace ▾

Encoding: windows-1252

Size: 0 bytes



[Download](#)

Response Message

[View](#)

XML ▾

Encoding: UTF-8

Size: 366 bytes

```
<env:Envelope>
  <env:Body>
    <ns1:AddRecordResponse>
      <ns1:Result>
        true
      </ns1:Result>
      <ns1:Reason>
        Awaiting authorization
      </ns1:Reason>
      <ns1:Identifier>
        84984-6249
      </ns1:Identifier>
    </ns1:AddRecordResponse>
  </env:Body>
</env:Envelope>
```

[Download](#)

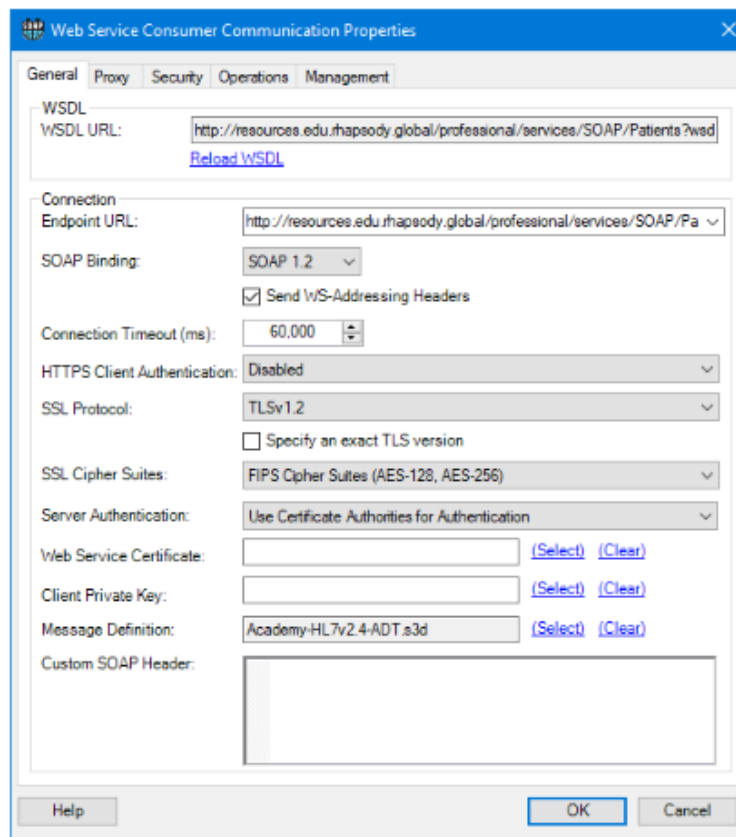
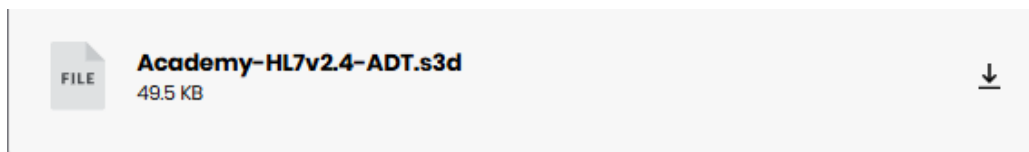
Extension - Working With HL7 Messages

For the final part of this exercise, we will send an HL7 ADT^A29 (delete person information) message to a Web Service Client communication point and use an associated message definition to extract fields from the message into the SOAP request. The response will be configured to update a message property with the response from the Web Service.

Start by creating a new SOAP Web Service Client communication point and use the previously used WSDL

(here: <http://resources.edu.rhapsody.global/professional/services/SOAP/Patients?wsdl> (opens in a new tab)) to configure it.

So that we can extract the patient identifier directly from the HL7 message, we need to add the *Academy-HL7v2.4-ADT.s3d* definition to the Message Definition property under the General tab.



Next, we need to add the DeleteRecord operation to the communication point.

The screenshot shows the 'Operation Editor' window. The 'Operation' section is configured with the following details:

- Name:** DeleteRecord
- Namespace:** http://edu.rhapsody.global/professional/patient
- Mode:** Request-Response
- SOAP Action:** um:DeleteRecord
- Request WS-Addressing Action:** um:DeleteRecord
- Enable MTOM:** ☐

The 'Request' tab is selected. Below it, the 'Configure how the outgoing SOAP request is built from the Rhapsody message.' section shows:

- Mode:** Construct SOAP body contents using a template and values from the Rhapsody message body and properties
- Template:**

```

1 <pat:DeleteRecord xmlns:pat="http://edu.rhapsody.global/professional/patient">
2   <pat:Identifier>string</pat:Identifier>
3 </pat:DeleteRecord>

```

The 'Content Insertion' section contains a table with the following columns: Source Type, Property / Field Name, Content Type, and Target XPath. A new row has been added with the following values:

Source Type	Property / Field Name	Content Type	Target XPath
Field	PID/PatientIdentifierList[0]/IDNumber	Text	/pat:DeleteRecord/pat:Identifier

The 'Signing / Encryption' section is collapsed, showing a link to 'Edit Signing and Encryption Parameters' and a note: 'The signing and encryption options are only available when using the asymmetric or symmetric security bindings.'

Buttons at the bottom include 'Help', 'OK', and 'Cancel'.

Under the **Content Insertion** section, add a new row into the table. This will add the contents of the PID/PatientIdentifierList[0]/IDNumber field into the message sent to the SOAP service.

Field	Value
Source Type	Field
Property / Field Name	PID/PatientIdentifierList[0]/IDNumber
Content Type	Text
Target XPath	/pat:DeleteRecord/pat:Identifier

Select the **Response** tab and change the **Mode** to **Extract contents from SOAP message into Rhapsody message body and properties**. This allows us to select individual fields from the SOAP response message to return as message properties. We

won't update the message body, so the HL7 message sent to the communication point will be used.

From previous steps in this exercise, we can see that the response to a successful delete message will look like this:

```
<?xml version="1.0" encoding="utf-8"?>
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope"
  xmlns:ns1="http://edu.rhapsody.global/professional/patient">
  <env:Body>
    <ns1:DeleteRecordResponse>
      <ns1:Result>true</ns1:Result>
      <ns1:Reason>Awaiting authorization</ns1:Reason>
    </ns1:DeleteRecordResponse>
  </env:Body>
</env:Envelope>
```

We want to return the result and reason fields as message properties. This will let us determine the outcome of the delete request and generate an appropriate HL7 acknowledgement message. The **Content Extraction** section of the **Response** tab needs to be updated with the following items:

Source XPath	Target Type	Target Property/Field
//ns1:DeleteRecordResponse/ns1:Result	Property	DeleteResult
//ns1:DeleteRecordResponse/ns1:Reason	Property	DeleteReason

Operation Editor

Operation

Name: Namespace:

Mode: SOAP Action:

Request WS-Addressing Action: ☐ Enable MTOM

Request Response Fault

Configure how the incoming SOAP response is used to populate the Rhapsody message.

Mode:

XML Namespace Mapping

Prefix	Namespace URI
ns1	http://edu.rhapsody.global/professional/patient
*	

Content Extraction

Source XPath	Target Type	Target Property / Field
//ns1:DeleteRecordResponse/ns1:R...	Property	DeleteResult
//ns1:DeleteRecordResponse/ns1:R...	Property	DeleteReason
*		

Signing / Encryption

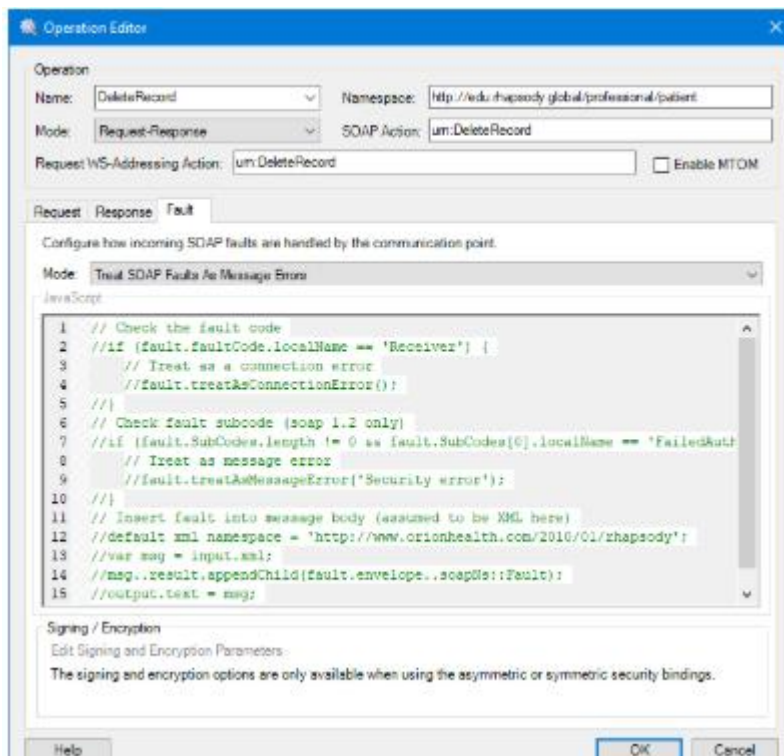
Edit Signing and Encryption Parameters

The signing and encryption options are only available when using the asymmetric or symmetric security bindings.

Help OK Cancel

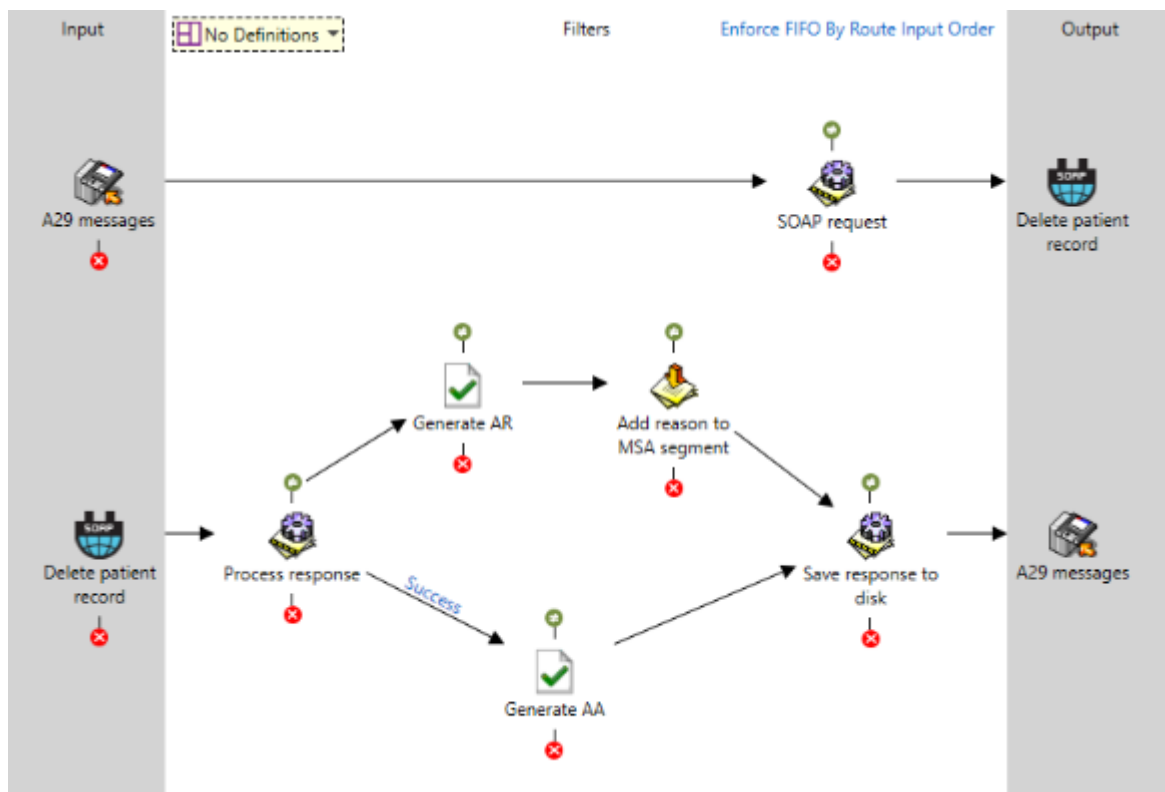
Select the **Fault** tab and change the **Mode** to **Treat SOAP Faults As Message Errors**.

This will send any message that returns a SOAP fault to the error queue. In a production environment, if there was an expectation that faults could be generated, they should be managed without relying on the error queue.



Create Route

Create a new route that is configured as follows:



- The conditional connector of the **Process response** no-operation filter should check that the DeleteResult message property contains a value of true.
- The two HL7 Acknowledgement Generation filters generate the appropriate type of HL7 acknowledgement message.
- The Content Population Filter adds the DeleteReason message property to the MSA.TextMessage field in the HL7 message.

Testing

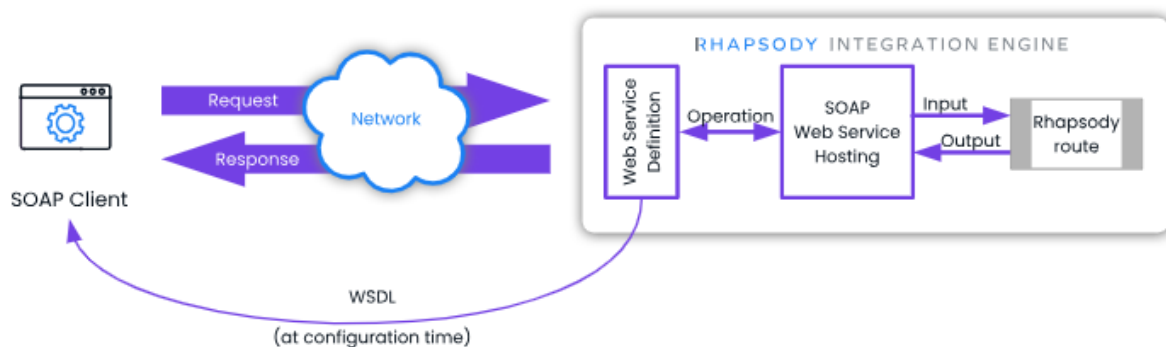
Download the set of messages. Process each of the test messages through the route. Observe the acknowledgement message that has been generated for each test message.

What happens if we send a different type of ADT message (say, an ADT^A28 - add person information message) to the communication point? What happens if we send a different type of HL7 message not supported by the message definition to the communication point?

SOAP Web Service Hosting

Introduction

The diagram below illustrates how the web service Definitions and the SOAP Web Service Hosting communication point work together in Rhapsody to serve SOAP requests received from external systems. The external system could be any system that understands SOAP, including another Rhapsody server.



SOAP Requests are received in Rhapsody and checked against the **Web Service Definition**. If a request does not comply with the definition, a SOAP fault is sent back to the client. Where the request is good, Rhapsody looks for a SOAP Web Service Hosting

communication point that has been configured to process the received SOAP **Operation** and sends the request to the communication point. If no communication point has been configured to process the operation, a fault is returned to the client. The configuration of the communication point determines how the request is input to the Rhapsody route where it will be processed. Once the Rhapsody configuration has processed the message, the **Response** message is constructed and sent back to the client via the SOAP Web Service Host communication point.

The process for creating a web service host in Rhapsody can be summarized as follows:

1. 1

Create the SOAP Web Service definition; this defines the format of the Request message and the format of the Response message which the service utilizes.

2. 2

Create a SOAP Web Service Hosting communication point to expose the service. This service accepts the requests and transmits the responses.

3. 3

Attach the SOAP Web Service Hosting communication point to the route which implements the logic required by the service.

Creating a New Web Service Definition

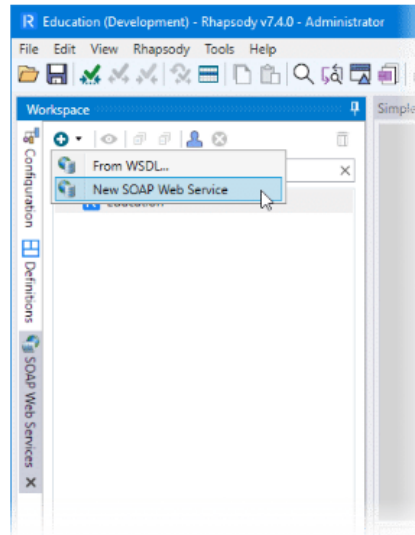
SOAP Web Service definitions hosted by Rhapsody are managed through the **SOAP Web Services** tab in the Rhapsody IDE Workspace. This is enabled from the **View** menu or **Managers** drop-down. Any SOAP web service that will be hosted by Rhapsody will need to be set up from this screen.

No configuration is required here when Rhapsody is used to connect to a web service hosted by another system (that is, when the SOAP Web Service Client is used).

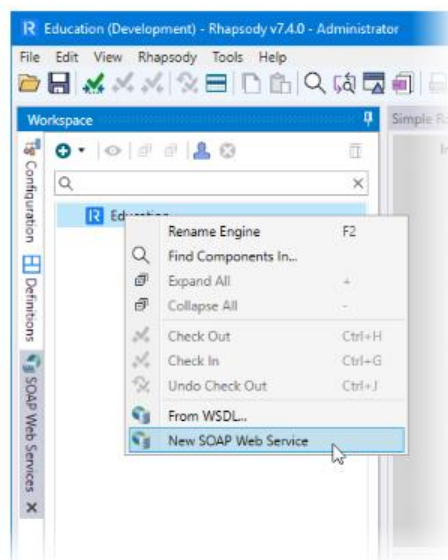
Web Service definitions can be created in one of two ways:

- From a WSDL: Use this approach if you have been provided with a WSDL that contains a definition for the web service interface that Rhapsody needs to host.
- By specifying all the settings manually: Use this approach if you are creating a new web service from a specification.

To create a new web service in Rhapsody, either select the **Add** button on the Workspace ribbon or right-click in the Workspace. If **From WSDL** is selected, Rhapsody prompts for the WSDL containing the web service definition. If **New SOAP Web Service** is selected, Rhapsody will create a new web service without performing any automatic configuration.



Adding a Web Service definition using the add button



Adding a Web Service definition using the right-click menu



The **SOAP Web Services Properties** screen is displayed in both cases. This screen has five tabs whose options enable a web service configuration to be added or edited. The following pages will detail the configuration required on each of these tabs.

The same **SOAP Web Services Properties** options will be displayed whether a web service is being added or edited.

General

The **General** tab of the **SOAP Web Service Properties** screen contains the bulk of the configuration for the web service. For many basic web services, all settings required to get the web service setup will be on this tab.

The screenshot shows the 'SOAP Web Services Properties' dialog box with the 'General' tab selected. The dialog has four tabs: General, Security, Operations, Management, and WSDL. The 'SOAP Web Service Information' section contains three fields: 'Service Name' (PatientLookupService), 'Service Namespace' (http://academy.rhapsody.health/Rhapsody/Professional/Laboratory), and 'Service Description' (Lookup patient details based on either name or identifier). The 'Http(s) Settings' section includes 'Protocol' (HTTP), 'SSL Protocol' (SSL), 'SSL Cipher Suites' (Strong cipher suites (AES-128, AES-256)), 'Local Address' (empty), 'Port' (8880), 'Private Key' (empty), and checkboxes for 'SOAP 1.1 Binding' (unchecked) and 'SOAP 1.2 Binding' (checked). The 'WS-Addressing' section has 'WS-Addressing Support' (Optional) and 'Asynchronous Requests' (Prohibited). At the bottom are 'OK', 'Cancel', and 'Help' buttons.

The screen is separated into several sections, which each deal with an element of the web service configuration.

SOAP WEB SERVICE INFORMATION

HTTP(S) SETTINGS

WS-ADDRESSING

The **SOAP Web Service Information** section contains basic information about the service.

- The **Service Name** is the name of the service within the WSDL and SOAP request. It should be chosen so that it describes the service provided or function performed by the web service.
- A **Service Namespace** should be entered. This is a URL that identifies the XML namespace for the body of the SOAP request.
- The **Service Description** needs to describe the purpose of the web service in a way that is useful to anyone else who will review the configuration in Rhapsody.

**SOAP WEB SERVICE
INFORMATION**

HTTP(S) SETTINGS

WS-ADDRESSING

The **HTTP(S) Settings** section details how a client connects to the Web Service.

- The **Protocol** will be either HTTP or HTTPS when a secure connection is required. If HTTPS is selected, a **Private Key** needs to be selected from the Certificate Store to enable Rhapsody to establish an SSL connection to the client.
- The **Port** number is the TCP port that Rhapsody will host the WSDL and Web Service requests on. The port number will need to be chosen so that it does not clash with the port number assigned for any other process running on the Rhapsody server or communications point running in Rhapsody.
- **Local Address** only needs to be specified if the server is connected to multiple networks and there is a requirement that the Web Service is only available from one.
- The **SOAP Binding** settings will be determined by the version of the SOAP protocol supported by any client application that will connect to the Web Service.

**SOAP WEB SERVICE
INFORMATION**

HTTP(S) SETTINGS

WS-ADDRESSING

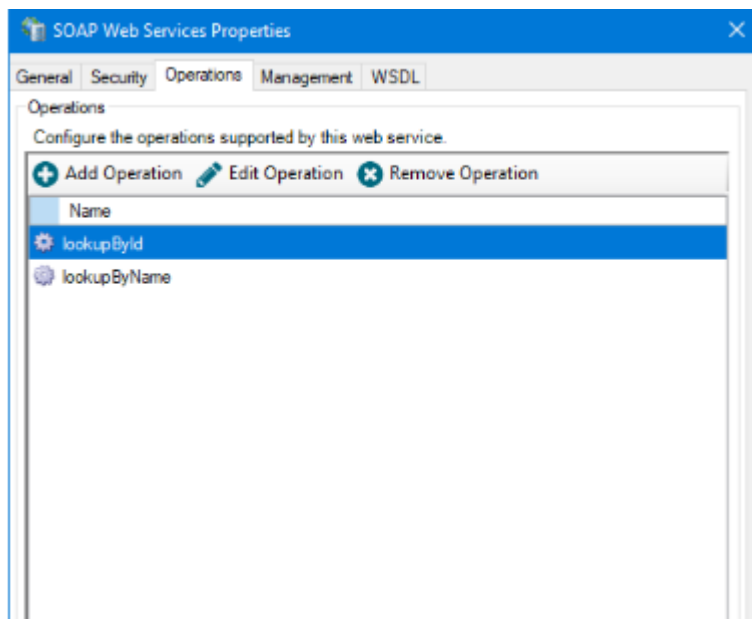
The **WS-Addressing** section contains settings for the setup of Web Service Addressing. For most Web Services hosted by Rhapsody, the default configuration will be correct.

- The **WS-Addressing Support** option determines if WS-Addressing directives are permitted in the SOAP message header.
- The **Asynchronous Requests** option determines if this Web Service will support asynchronous requests from clients.

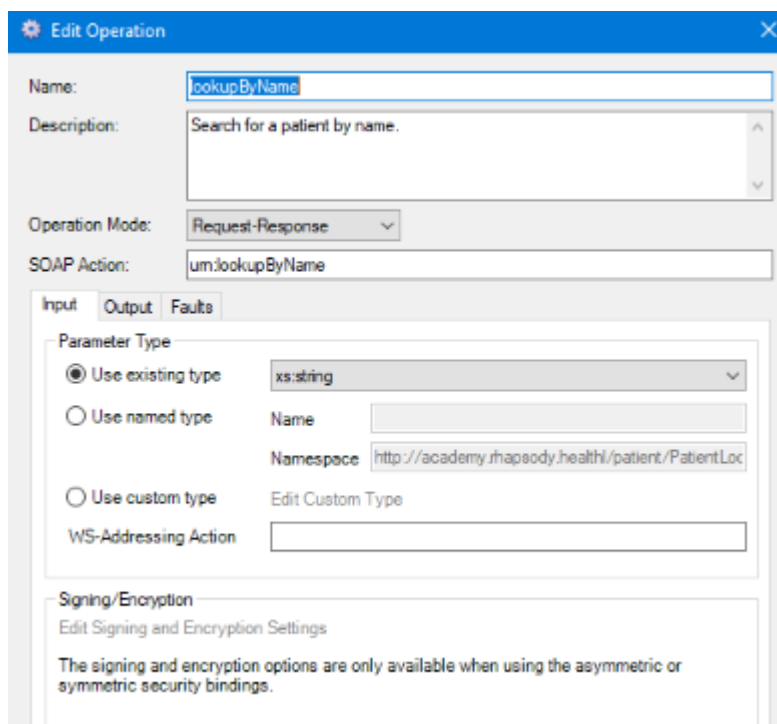
Operations

For the web service to be useful, one or more operations must be defined. These define the request and response structure.

Add an operation by selecting **Add Operation** on the **Operations** tab.



This will display the **Edit Operation** screen.



The operation needs a **Name** and **Description**. The name should indicate the action that will be performed when the web service is called.

Set the **Operation Mode** to **Request-Response** if the operation needs to return something to the client, such as a patient search, or **In-Only** if the client does not expect any feedback on the operation, like an audit update.

The **SOAP Action** field defines the action this operation will perform (in this case urn:lookupByName). If the field is not populated, Rhapsody will automatically choose the default value.

Parameters

This section defines what parameters the operation will accept in the SOAP body. A client system may look at the parameters as they are published in the WSDL to configure the request and help in the interpretation of the response messages.

There are three sets of parameters: **Input**, **Output** and **Faults**. The first of these will always be selectable. The other two will only be selectable when the Operation Mode is set to Request-Response. The content of each tab looks the same.

There are three options for the parameters:

- **Use existing type** - indicates that the body will contain a single parameter of the indicated type. The two special values of Empty Type and Any Type are used when no data is expected or data containing any valid XML is permitted.
- **Use named type** - allows the web service to be configured to use the named XML variable in the specified namespace.
- **Use custom type** - permits simple parameter types to be defined for this operation. Select the **Edit Custom Type** link to display the Edit Custom Types window.

Type Name (leave blank for default):

☐ Allow additional parameters not listed below

	Name	Type	Required	Repeating
▶	GivenName	xs:string	☐	☒
	FamilyName	xs:string	☐	☐
*			☐	☐

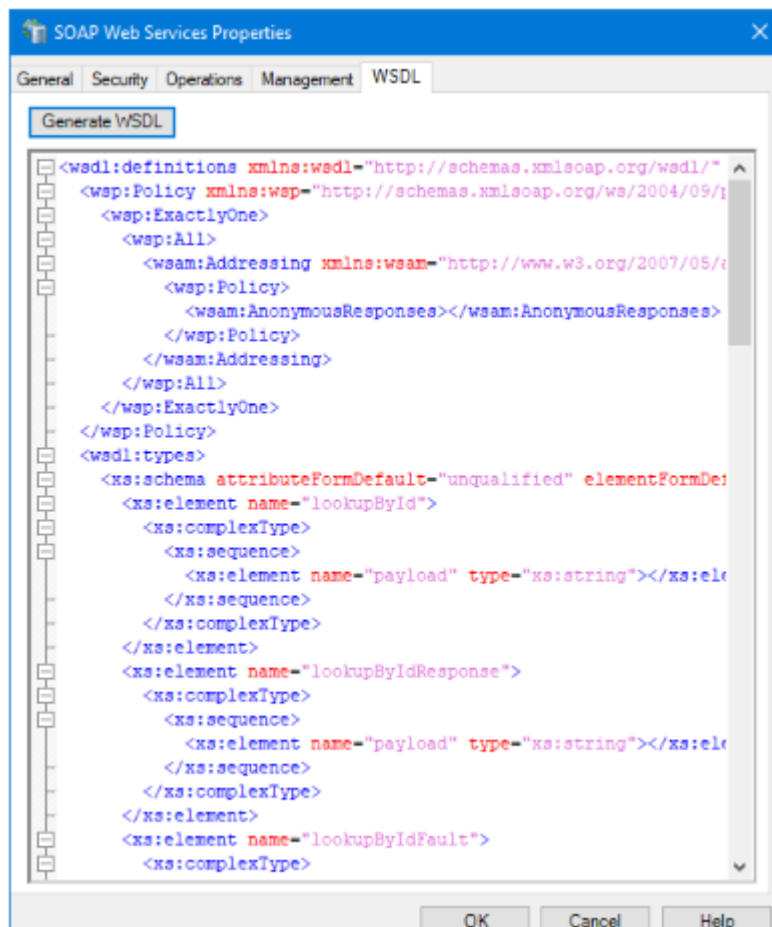
OK Cancel

WSDL

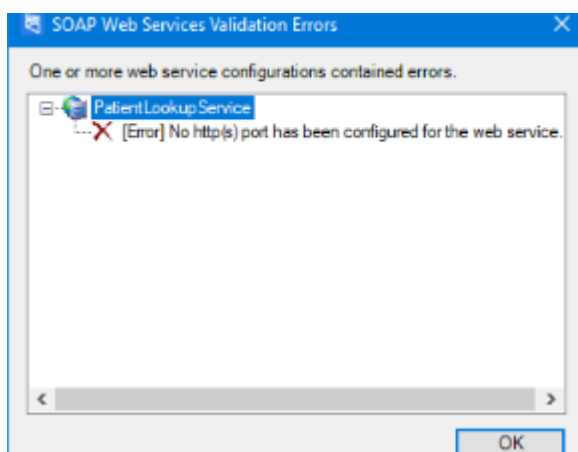
WSDL stands for Web Service Description Language. It is an XML file that details how a web service client should interact with the host. The WSDL includes details of all accepted operations and their parameters. The file also contains details of the URL that must be accessed to perform the web service request.

Generating the WSDL

As the last action to be performed when configuring the web service, the **WSDL** tab should be selected. Select **Generate WSDL** and wait for generation to complete. Ensure the WSDL definition appears onscreen.



If an error is encountered generating the WSDL, details will appear in a popup window. Fix these before you attempt generation again.



Accessing the WSDL

Most web service clients use the WSDL to configure their requests by downloading the WSDL. The WSDL for any web service defined in Rhapsody will be available for download at the following URL:

`http://<hostname>:<port>/services/<webservice>?wsdl`

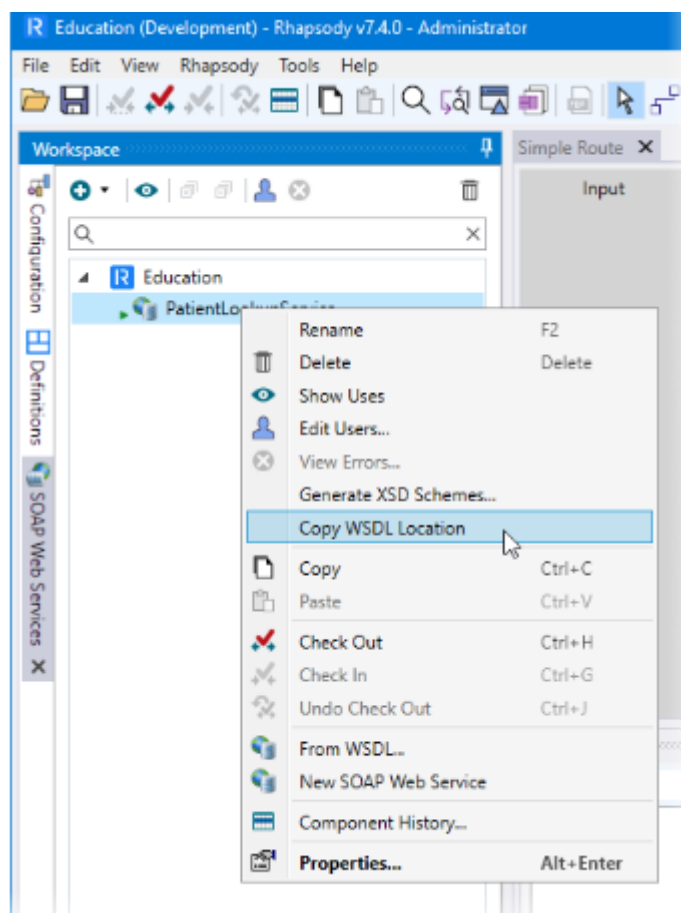
Where:

- <hostname> is the hostname of the Rhapsody sever.
- <port> is the port number the web service has been configured to listen on.
- <webservice> is the name of the web service in Rhapsody.

For example, for a web service called PatientLookService, running on port 8880, the URL to access the WSDL would be:

`http://localhost:8880/services/PatientLookupService?wsdl`

A link to the WSDL is also available for download from **SOAP Web Services** Workspace tab in the Rhapsody IDE by right-clicking on the name of the service, or from the **Web Services** screen in the **Management Console**.



SOAP Web Service Hosting Communication Point

The SOAP Web Services Hosting communication point hosts a SOAP web service that has been defined with a Web Services Definition. The following communication modes are supported:

- Input
- In -> Out

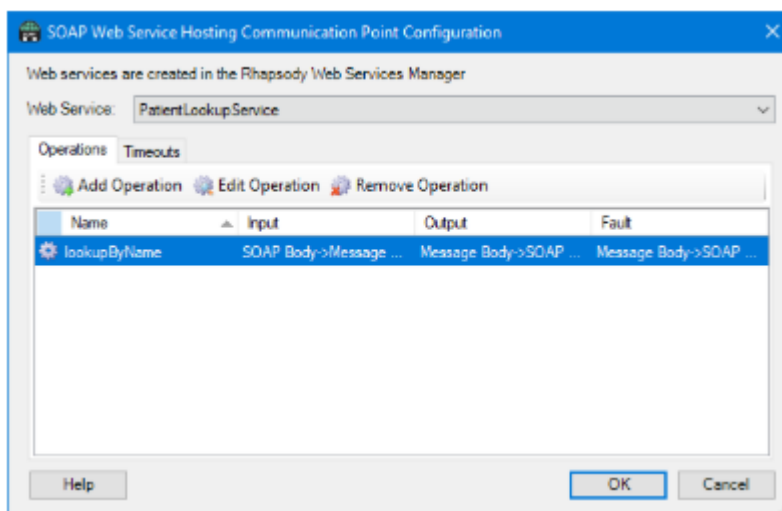
The **Configuration** tab for the SOAP Web Service Host shows a summary of the configuration. To configure the communication point, select the **Edit Communications Point Configuration** button. This will display the **SOAP Web Service Hosting Communications Point Configuration** screen.

The **Web Service** drop-down list lists the web services configured in Rhapsody. The appropriate web service must be selected from the list.

Selecting **Add Operation** adds a web service operation to the communications point. If the web service has been configured with multiple operations, it is possible for a single SOAP Web Service Host communications point to process all of them.

Operations

For each defined operation, the **Operations Parameters Mapping** screen defines how input, output, and fault messages are transformed between a SOAP message in the web service and a message on the Rhapsody route.



Operation Parameter Mapping - lookupByName

Input Output Fault

Configure how incoming SOAP messages are transformed into Rhapsody messages.

Mode: Extract contents from SOAP message into Rhapsody message body and properties

XML Namespace Mapping

Prefix	Namespace URI
ns1	http://academy.rhapsody.health/patient/PatientLookupService
*	

Content Extraction

Source XPath	Target Type	Target Property Name
*		

Help OK Cancel

In all cases it is possible to use the SOAP body, the SOAP envelope as the body, or use Content Extraction (for the input) or Template (for output and fault). When doing content extraction from the SOAP message on the **Input** tab, remember to take note of the prefixes that have been defined for the XML namespaces associated with the SOAP definition.

Message Properties

The following message properties are added to all messages received via a SOAP Web Service Hosting communications point.

Property	Description
rhapsody:InputSoapEnvelopeNamespace	The namespace of the SOAP envelope.
rhapsody:InputWebService	The name of the Rhapsody web service the message was received on.
rhapsody:InputWebServiceOperation	The name of the operation the message was sent to.
rhapsody:WebServiceCallerIPAddress	The IP Address of the web service client.

In addition, any WS-Addressing headers sent by the client will be added as message properties with a wsa prefix.

Reporting an Error

If an error occurs while processing a message, it can be desirable to report this to the client. This is done by setting the rhapsody:SoapFault property on the message and configuring the **Fault** tab.

The screenshot shows the 'Operation Parameter Mapping - lookupByName' dialog box with the 'Fault' tab selected. The 'Mode' is set to 'Construct SOAP body contents using a template and values from the Rhapsody message body and properties'. The 'Template' section contains an XML snippet for a SOAP fault. The 'Content Insertion' table lists two properties: 'ErrorDesc' and 'ErrorLocation', both mapped to the 'Text' content type and the target XPath '/soapenv:Fault/soapen...'. The dialog includes 'Reload Template' and 'View Template' buttons, and 'OK' and 'Cancel' buttons at the bottom.

Operation Parameter Mapping - lookupByName

Input Output Fault

Configure how outgoing Rhapsody messages are transformed into SOAP faults.

Mode : Construct SOAP body contents using a template and values from the Rhapsody message body and properties

Template

```
1 <soapenv:Fault xmlns:soapenv="http://www.w3.org/2003/05/soap-envelope">
2   <soapenv:Code>
3     <soapenv:Value>soapenv:Sender</soapenv:Value>
4     <soapenv:Subcode>
5       <soapenv:Value>subcode</soapenv:Value>
6     </soapenv:Subcode>
7   </soapenv:Code>
```

Reload Template View Template

Content Insertion

	Source Type	Property Name	Content Type	Target XPath
▶	Property	ErrorDesc	Text	/soapenv:Fault/soapen...
	Property	ErrorLocation	Text	/soapenv:Fault/soapen...
*				

Help OK Cancel

SOAP Web Service Hosting

It is possible to monitor all web services that have been configured in Rhapsody via the **Monitoring > Web Services** screen in the Management Console.

The screenshot shows the 'Web Services (1)' screen in the Rhapsody Management Console. The left sidebar contains a navigation menu with 'Monitoring' selected, and 'Web Services' highlighted under the 'Monitoring' section. The main area displays a table of web services. One service, 'PatientLookupService', is listed as 'Running' on port 8880 using HTTP. The table includes columns for status, action, port, protocol, received, sent, uptime, idle time, and WSDL.

R Education 7.4.0 (Development) Administrator 16-Jul-2024 15:55 Rhapsody INTEGRATION ENGINE

Monitoring

- Engine Monitor
- Communication Points
- Routes
- Web Services
- REST Clients
- System Monitor
- Error Queue
- Hold Queue
- Ports in Use

Find Messages

Notifications

Management

System State

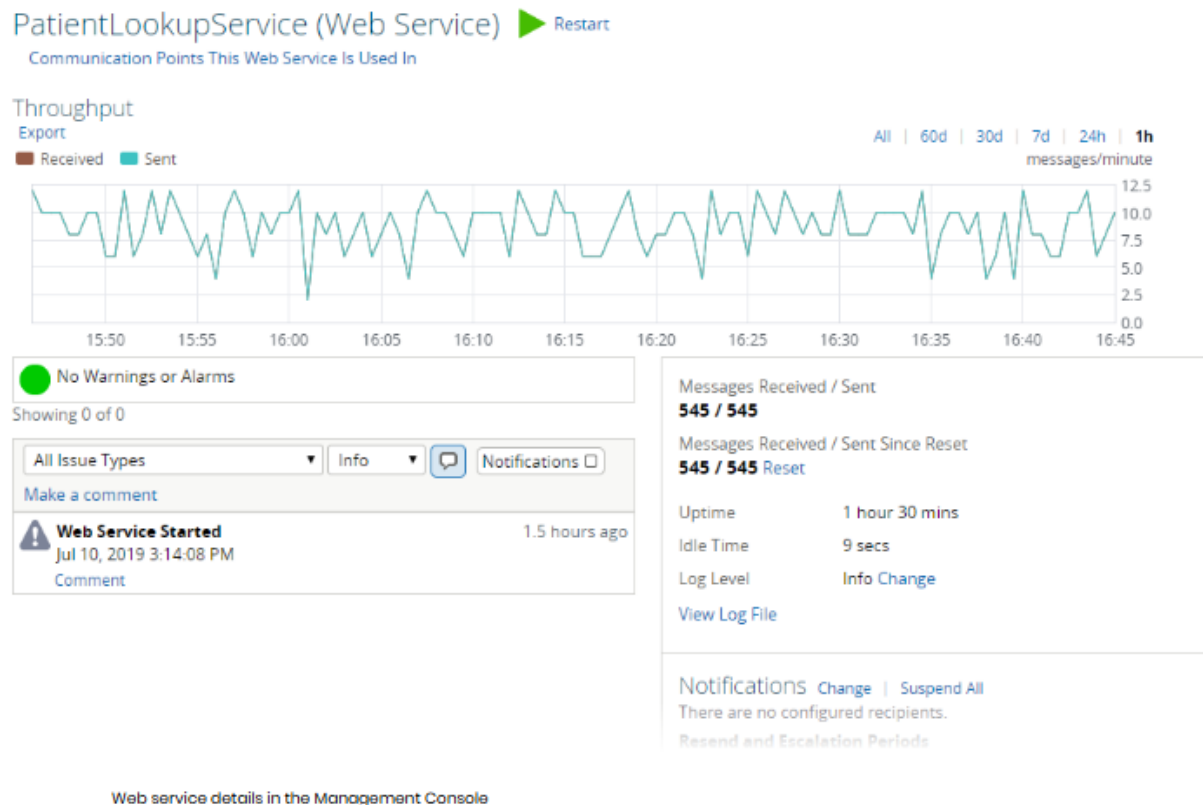
Reports

Web Services (1)

▶ Running (1) □ Search by name

WEB SERVICE	STATUS	ACTION	PORT	PROTOCOL	RECEIVED	SENT	UPTIME	IDLE TIME	WSDL
▶ PatientLookupService	Running	Restart	8880	HTTP	145	145	44 mins	1 sec	WSDL

Selecting the name of the web service displays a monitoring page for the selected web service that presents the same information as is available for all communications points configured in Rhapsody. Selecting the **WSDL** link will open the WSDL for the web service in a new browser window. This could be used to get the URL for the web service for an external system, or to download the WSDL for a system that is unable to fetch the file directly.



Exercise - SOAP Web Service Hosting

Introduction

In this exercise, we will create a new SOAP web service in Rhapsody from a specification. At the end of the exercise, a simple test will then performed to confirm that the web service is configured correctly and is able to process requests.

The SOAP web service to be implemented is designed to order a laboratory test. The web service will have a name of LaboratorySystem. The service does not need to support secure connections. Only Soap 1.2 bindings are required.

The web service will support one operation, OrderTest which operates in request-response mode. The operation has two required parameters:

- patientId - This is the patient identifier of the patient that the order is being performed on.

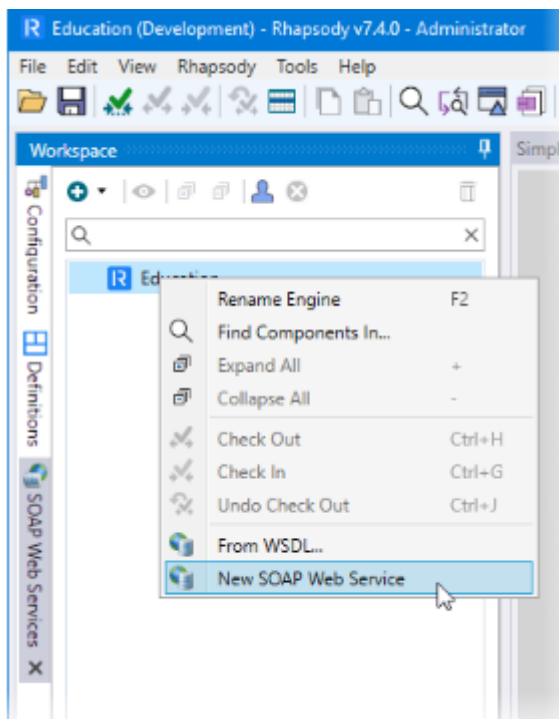
- **testCode** - This is a repeating parameter that contains a code for each test to be performed.

The operation will respond with a single parameter:

- **orderNumber** - This is a field that contains the set of all order numbers for the requested order.

Create a New Web Service Definition

From the **Rhapsody IDE**, open the **SOAP Web Services** Workspace tab. Select then engine name, then right-click and select **New SOAP Web Service** from the drop-down list.



Configuration

On the **General** Tab, enter the details for the web service from the specification. It should be configured with the following details:

Field	Value
Service Name	LaboratorySystem
Service Namespace	http://academy.rhapsody.health/Rhapsody/Professional/Laboratory
Protocol	HTTP
Port	8880

The screenshot shows the 'SOAP Web Services Properties' dialog box with the 'General' tab selected. The 'Web Service Information' section contains the following fields:

- Service Name: LaboratorySystem
- Service Namespace: http://academy.rhapsody.health/Rhapsody/Professional/Laboratory
- Service Description: Laboratory test system

The 'Http(s) Settings' section contains the following fields:

- Protocol: HTTP
- SSL Protocol: SSL
- SSL Cipher Suites: Strong Cipher Suites (AES-128, AES-256)
- Local Address: (empty)
- Port: 8880
- Private Key: (empty) with links for (Select) and (Clear)
- ☐ SOAP 1.1 Binding
- ☒ SOAP 1.2 Binding

The 'WS-Addressing' section contains the following fields:

- WS-Addressing Support: Optional
- Asynchronous Requests: Prohibited

At the bottom of the dialog are buttons for OK, Cancel, and Help.

The value for the **Service Namespace** was chosen because the organization that authored this web service is academy.rhapsody.health and /Rhapsody/Professional/Laboratory represents the type of information provided by the service. This detail is also reflected in the **Service Name**.

The **Port** number was chosen as we have not previously used it in any exercises, so it should be available to use.

OrderTest Operation

Under the **Operations** section, select **Add** to add a new operation. Enter the operation details from the specification.

The Operation will need to have the following details:

Field	Value
Name	OrderTest
Operation Mode	Request-Response
SOAP Action	urn:OrderTest <i>(this will be automatically created for you when you save your changes)</i>
Input Parameter Type	Custom Type
Output Parameter Type	Custom Type

Edit Operation

Name: OrderTest

Description: Order a laboratory test

Operation Mode: Request-Response

SOAP Action:

Input Output Faults

Parameter Type

☒ Use existing type xs:string

☐ Use named type Name Namespace http://academy.rhapsody.health/Rhapsody/Profess

☐ Use custom type Edit Custom Type

WS-Addressing Action

Signing/Encryption

Edit Signing and Encryption Settings

The signing and encryption options are only available when using the asymmetric or symmetric security bindings.

OK Cancel

For both the input and output parameters, you will need to select **Use custom type** and select the **Edit Custom Type** link to enter the parameters.

Type Name (leave blank for default):

☐ Allow additional parameters not listed below

	Name	Type	Required	Repeating
▶	patientId	xs:string	☒	☐
	testCode	xs:string	☒	☒
*			☐	☐

OK Cancel

Input Setting

Type Name (leave blank for default):

☐ Allow additional parameters not listed below

	Name	Type	Required	Repeating
▶	orderNumber	xs:string	☒	☐
*			☐	☐

OK Cancel

Output Settings



Create WSDL

Finally, select the **WSDL** tab and ensure the WSDL is generated and displayed on screen. Save and **Check In** your changes.

Create SOAP Web Service Hosting Communication Point

Create a new SOAP Web Service Hosting communications point. Configure the new communications point as follows:

1. Connection Tab:

Change the **Connection Mode** to In->Out.

The screenshot shows the 'Process LaboratorySystem requests Properties' dialog box with the 'Connection' tab selected. The 'Connection Properties' section has 'Connection Mode' set to 'In->Out' and 'Continue routing' checked. 'Number of Connections' is 1 and 'Idle Timeout (ms)' is 0. 'Outgoing Messages' is set to 'Send via Original Connection (Discard if original connection has closed)'. The 'Connection Retries' section has 'Retry Type' set to 'Linear', 'Initial Retry Delay (ms)' is 5000, 'Number of Retries' is 5, 'Maximum Retry Delay (ms)' is 300000, 'Retries per Message' is 'Infinite', and 'Message Retry Delay (ms)' is 0. 'Retries Exceeded Action' is 'Shut down communication point', 'Redirect Target' is 'Select Communication Point', and 'Included Messages' is 'All Messages'. The 'In->Out Properties' section has 'Response Timeout (ms)' set to 60000, 'Discard message once timeout occurs' is unchecked, 'Retry Count' is 'Infinite', and 'Fail Action' is 'Treat as error'. Buttons for 'Help', 'OK', and 'Cancel' are at the bottom.

Connection tab for SOAP Web Service Hosting communication point

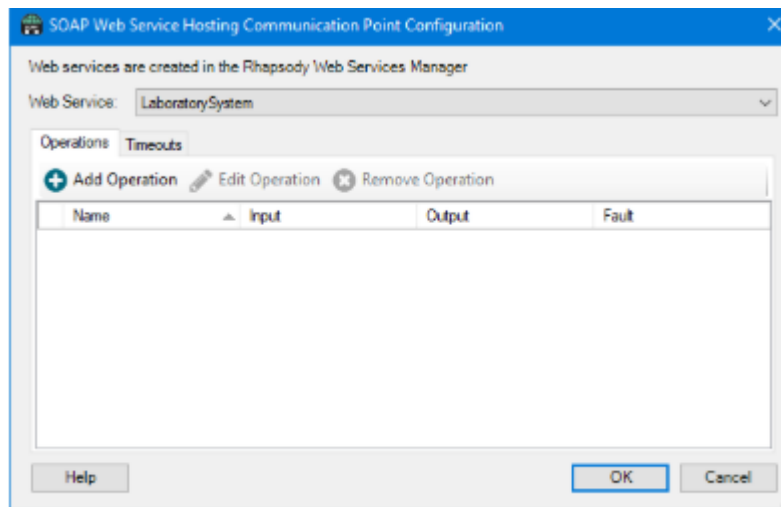
Configuration Tab

Select the **Edit Communications Point Configuration** link.

The screenshot shows the 'Process LaboratorySystem requests Properties' dialog box with the 'Configuration' tab selected. The 'Edit Communication Point Configuration' link is visible at the top. Below it, a yellow box displays the text 'Not Configured'. Buttons for 'Help', 'OK', and 'Cancel' are at the bottom.

Configuration tab for SOAP Web Service Hosting communication point

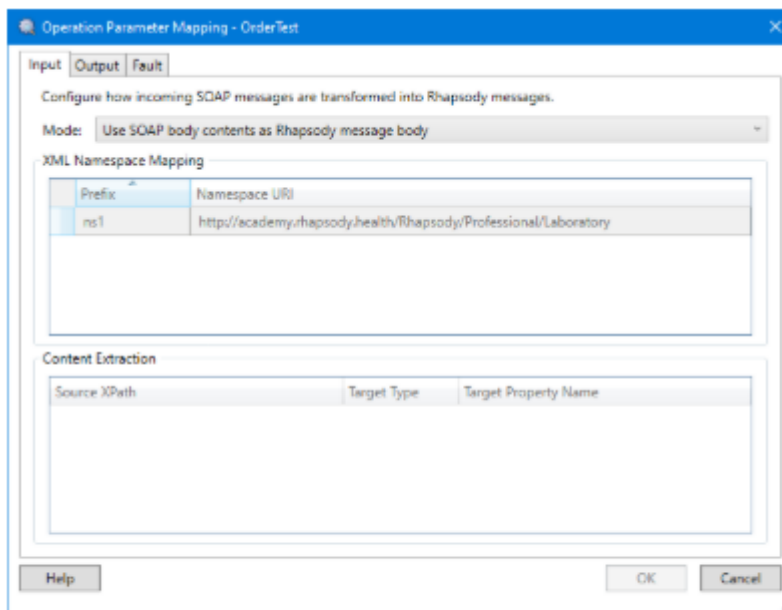
Select LaboratorySystem as **Web Service**.



SOAP Web Service Hosting Configuration

Select **Add Operation** and add the OrderTest operation.

The **Input** mode should be set to **Use SOAP body contents as Rhapsody message body**.



Operation Parameter Mapping - Input settings

The **Output** mode should be **Construct SOAP body contents using a template**.

Under **Content Insertion** the body should be added to the following XPath: `//lab:orderNumber`.

Operation Parameter Mapping - OrderTest

Input Output Fault

Configure how outgoing Rhapsody messages are transformed into SOAP responses.

Mode: Construct SOAP body contents using a template and values from the Rhapsody message body and properties

Template

```

1 <lab:OrderTestResponse xmlns:lab="http://academy.rhapsody.health/Rhapsody/Profession"
2   <lab:payload>string</lab:payload>
3 </lab:OrderTestResponse>

```

Content Insertion

Source Type	Property Name	Content Type	Target XPath
Body		Text	//lab:orderNumber

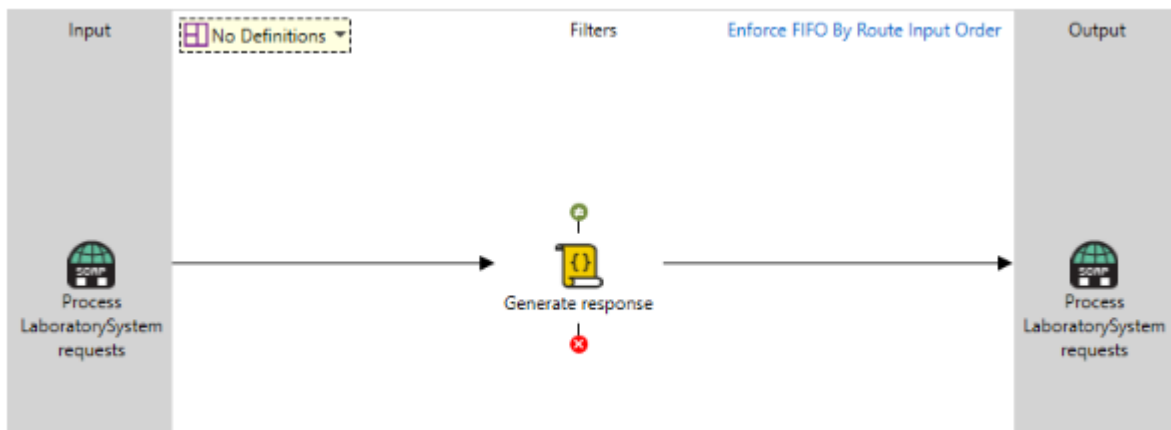
Help OK Cancel

Operation Parameter Mapping - Output setting

Once you have completed all the configuration, select **OK** to save your changes.

Create Route

Create a new route, then add the SOAP Web Service Hosting communications point you created as both an input and an output.



Route Layout

Add a JavaScript filter to the route and configure it as follows:

```
// Create the output message
```

```
const next = output.append(input[0]);
```

```
// Get patient identifier from SOAP request

const patientId = next.getField("//lab:patientId");


// Get set of orders from SOAP request

let orderedTests = "";

for (let i = 0; i < next.getRepeatCount("//lab:testCode"); i++) {

    orderedTests = orderedTests + "|" + next.getField("//lab:testCode[" + (i + 1) + "]");

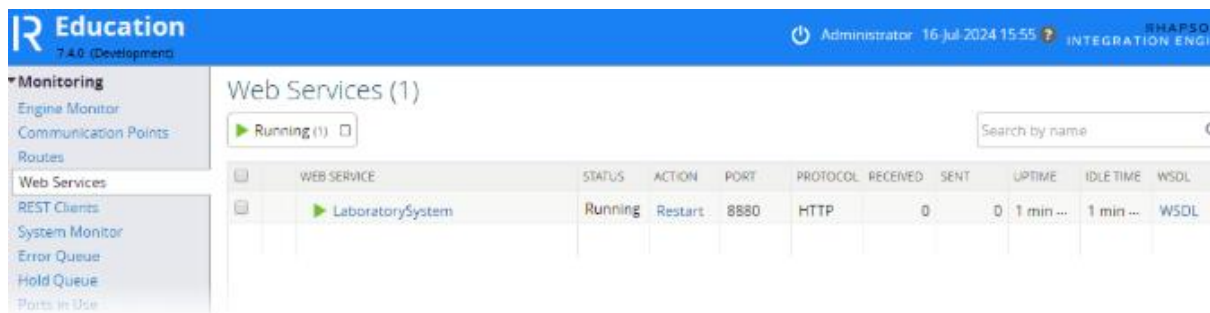
}


// Update message body with order number

next.text = patientId + orderedTests;
```

Testing

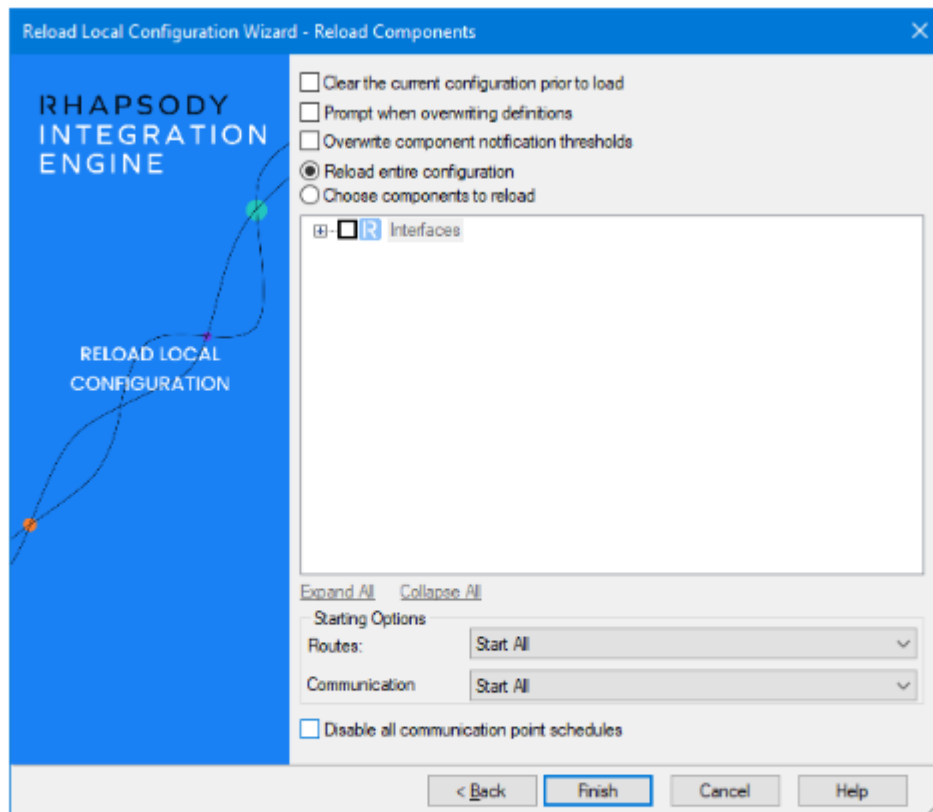
Log in to the Management Console and select **Monitoring > Web Services**.



Select on the **WSDL** link associated with the web service definition you created and confirm that an XML file is displayed in a new browser window.

Send a test message

Download the exercise RLC file and import it using the Rhapsody IDE.



When loading the configuration, select the options to **Start All** routes and communications points.

When loading the configuration, select the options to **Start All** routes and communications points.

Inspect Messages in Management Console

This configuration will send a message to the web service you have created once every 60 seconds. The response from the web service will appear on the Hold Queue. To view the response message, log into the Management Console and view the **Hold Queue**. Select the first message on the queue that has OrderTest WS as the component. The message should show the response from the web service.

Message Properties ▾

Property Name	Property Value
InputTime	1438119553811
rhapsody:BodyEncoding	UTF-8
rhapsody:InputCommunicationPoint	Generate test message
rhapsody:InputCommunicationPointId	5638
rhapsody:ResponseLatency	1749

Showing 5 of 5

Message Body ▾

XML ▾ Encoding: UTF-8 Size: 175 by

```
<lab:OrderTestResponse>
  <lab:orderNumber>
    CQ99432D|CBC|Hb1c
  </lab:orderNumber>
</lab:OrderTestResponse>
```

[Download](#)

Navigate to the SOAP Web Service Hosting communication point you created and look at the input queue to see the message as it was received and sent by the web service hosting route.

Finally, stop all the components under the *Rhapsody Professional/Working with Web Services/Exercise - Web Service Hosting* folder to stop further messages from being sent to the web service.

Testing

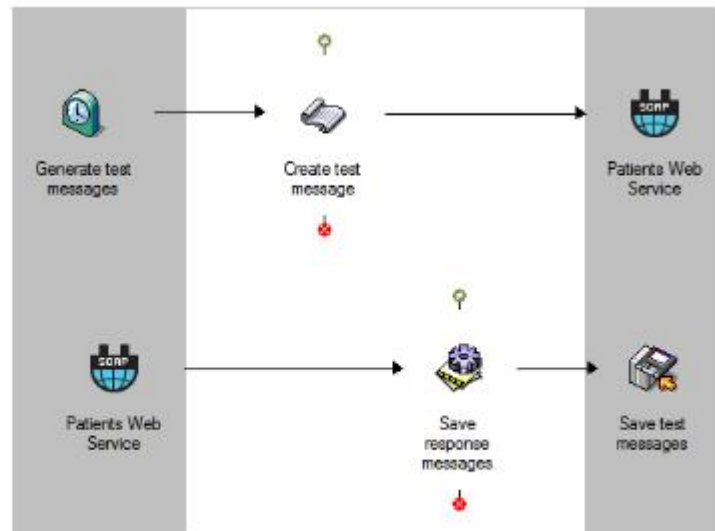
Introduction

There are many ways that web services, both external and those that you have created, can be tested. This lesson introduces a few popular methods for testing web services associated with Rhapsody. The tools presented here should not be considered a comprehensive list. Where there are specific testing needs, a search online will frequently reveal other testing options.

Rhapsody

The various REST and SOAP communication points in Rhapsody can be used to test the behavior of an internal or external web service. When using Rhapsody as a test tool, the communication point should be configured to return the raw response from the web service to the route. A simple route is then constructed around the communication point to modify each request as appropriate for testing being conducted. The exercises conducted as part of this module have used this approach to illustrate the data being returned by testing the REST and SOAP services.

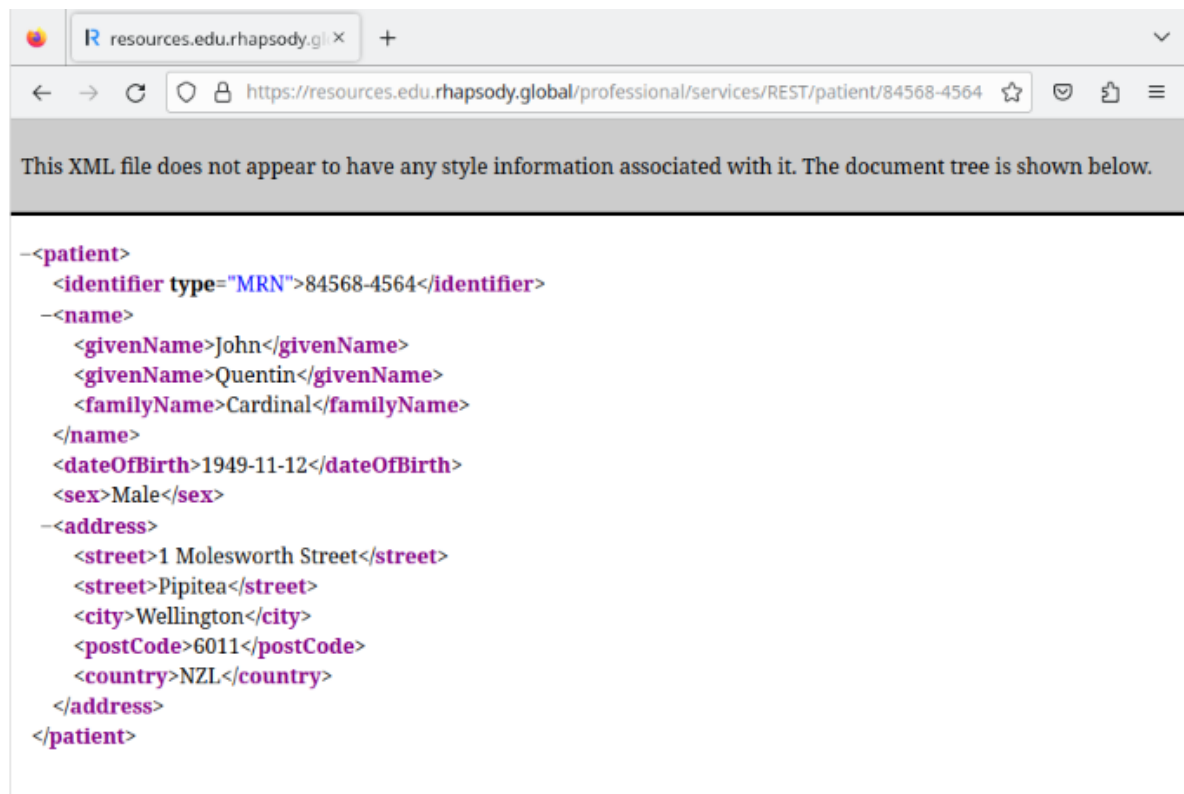
For example, to test a SOAP web service, the following route could be used to generate a stream of messages, modify them in a JavaScript filter then send them to the web service. The SOAP Web Service Client communication would be configured to return the entire SOAP envelope to the route, where it could be processed or stored for analysis.



A Rhapsody route being used to test a SOAP web service

Web Browser

A web browser can be used to perform a GET request on a resource with ease. However, using a web browser to perform other types of request either requires a REST testing plugin or a good working knowledge of HTML and JavaScript. All major web browsers also contain built in developer tools that are able to inspect all network traffic. This can be useful when needing to see the request and response headers associated with a request.



Testing of a REST web service in Firefox

Further Reading

- Chrome: <https://developer.chrome.com/devtools>(opens in a new tab)
- Edge <https://docs.microsoft.com/en-us/microsoft-edge/devtools-guide-chromium>(opens in a new tab)
- Firefox: https://developer.mozilla.org/en-US/docs/Tools/Network_Monitor(opens in a new tab)
- Safari: <https://developer.apple.com/safari/tools/>(opens in a new tab)
- Fiddler: <http://www.telerik.com/fiddler>(opens in a new tab)

cURL

cURL is a command line tool for performing requests to resources that use a URL syntax. This includes REST style HTTP requests. It can be used in testing to access specific URLs. cURL provides downloads for almost every conceivable operating system.

Useful Command Line Options

Option	Details
-X <method>	The HTTP method to be used. If not supplied, GET is assumed.
-d <data>	Include the specified data in the request. Where the data is in a file, is specified by prepending the filename with an @ sign.
-i	Include the complete response message (that is, headers and body) in the output.
-v	Verbose. Include request and response headers along with status updates.

Examples

Get Request

```
rhapsody@Testing:~$ curl
http://resources.edu.rhapsody.global/professional/services/REST/patient/87619-9902
<?xml version="1.0" encoding="UTF-8"?>
<patient>
  <identifier type='MRN'>87619-9902</identifier>
  <name>
    <givenName>Lila</givenName>
    <givenName>Betty</givenName>
    <familyName>Johnson</familyName>
  </name>
  <dateOfBirth>1991-12-12T08:30</dateOfBirth>
  <sex>Male</sex>
  <address>
    <street>221 Oxford Terrace</street>
    <city>Christchurch</city>
    <postCode>8011</postCode>
    <country>NZL</country>
  </address>
```

</patient>

Delete request

```
rhapsody@Testing:~$ curl -X DELETE
```

```
http://resources.edu.rhapsody.global/professional/services/REST/patient/87619-9902
```

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<message type='Information'>
```

```
  <code>202</code>
```

```
  <description>Delete awaiting authorization</description>
```

```
</message>
```

Put request

The submission data is taken from a file (the name of the file to be sent to the web service is prepended with an @ sign).

```
rhapsody@Testing:~$ curl -X PUT -d @UpdatePatient-2.xml
```

```
http://resources.edu.rhapsody.global/professional/services/REST/patient/87619-9902
```

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<message type='Information'>
```

```
  <code>202</code>
```

```
  <description>Update awaiting authorization</description>
```

```
</message>
```

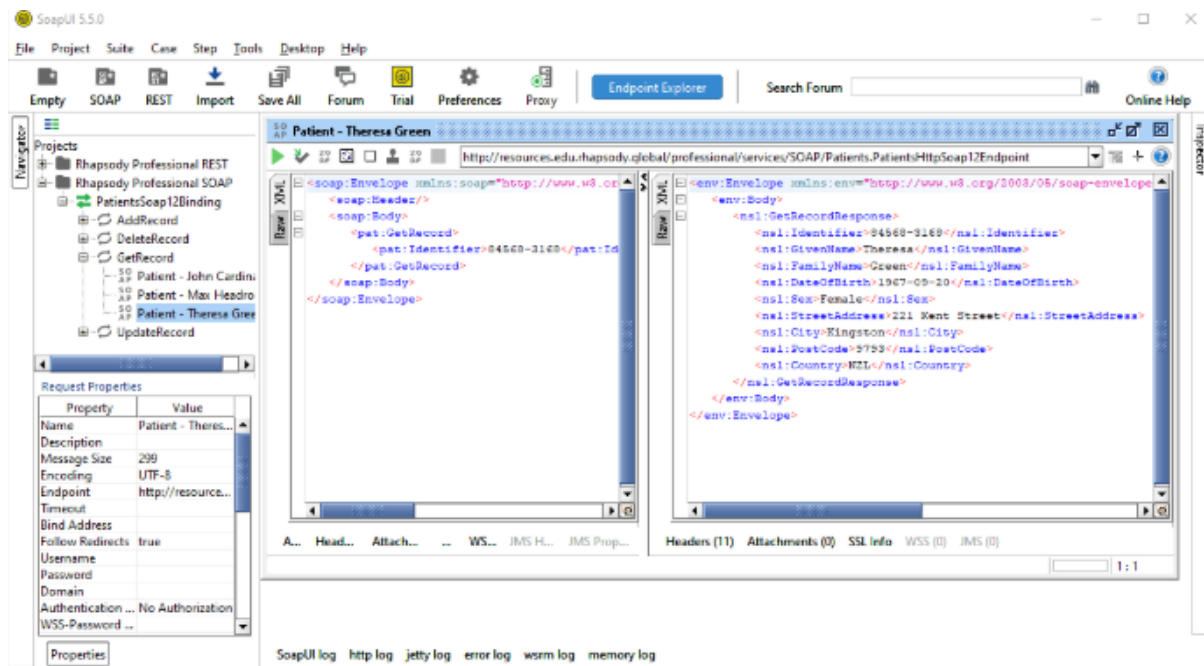
Further Reading

<https://curl.haxx.se>(opens in a new tab)

SoapUI

SoapUI is designed as a comprehensive web service testing application. While it can sometimes be difficult for new users to understand, it is a very powerful tool able to act as a complete test system for a Rhapsody web service configuration. It can be used to test either SOAP or REST web services. It is also able to act as a mock web service host

in place of a real system. Both free (open source) and paid versions of SoapUI exist. For the sorts of testing done in support of Rhapsody, the free version is adequate.



Testing a SOAP request in SoapUI

Further Reading

- <https://www.soapui.org/getting-started/introduction.html>(opens in a new tab)
- Testing SOAP: <http://www.soapui.org/soap-and-wsdl/getting-started.html>(opens in a new tab)
- Testing REST: <https://www.soapui.org/rest-testing/getting-started.html>